9 Requirements Engineering

Before you start to implement a software system, you need to understand what the system is intended to do. This intended functionality is the "Requirements". The process of creating these requirements is called Requirement Analysis or Requirement Engineering. It is the process of understanding what you want and what you need in your software.

Requirements Engineering (RE) refers to the process of formulating, documenting, and maintaining software requirements.

The results of the Requirement Analysis or Requirement Engineering process is normally one or more documents, called the Software Requirement Specification (shorted "SRS").

The requirements are in some cases created by the customer, at least the overall requirements (it defines "What" the customers want), while more details are normally created by architects and developers in the software company that is going to develop the actual software. Here we can have both "What" and "How" the software shall be designed or implemented.

The main challenges in Requirements Engineering are that the customers most often don't know what they want or are not qualified to know what they need.

In general, we can summarize the following:

- Stakeholders don't know what they want.
- Stakeholder's express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organizational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge, and the business environment may change.

In Figure 9-1 we see different types of requirements.

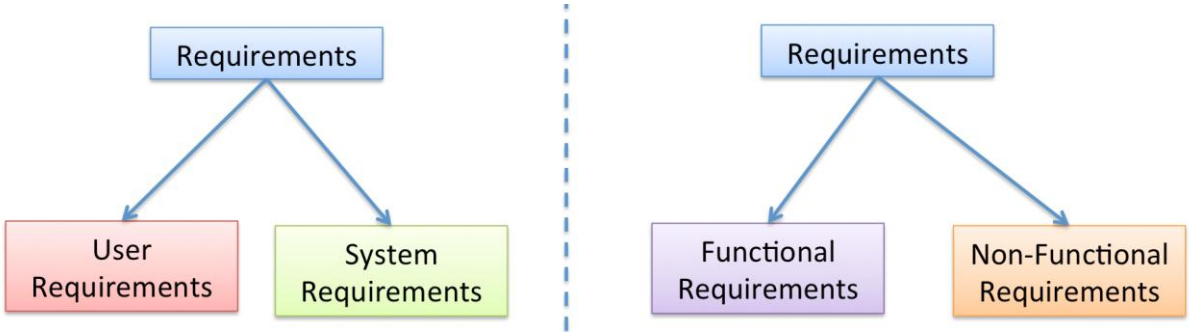


Figure 9-1: Software Requirements

In Figure 9-2 and below we explain the different software requirements categories in more detail.

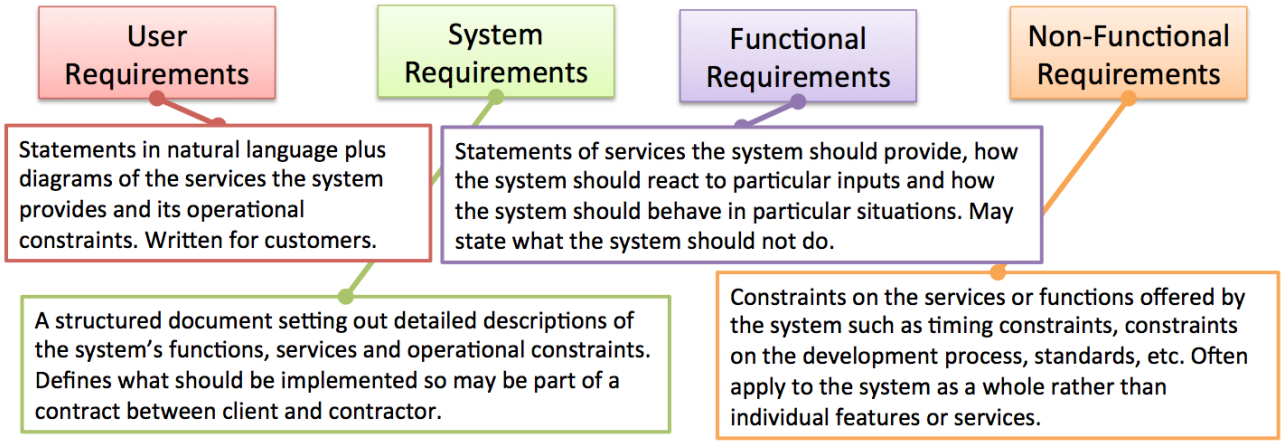


Figure 9-2: Software Requirements Categories Overview

User Requirements

Statements in natural language plus diagrams of the services the system provides and their operational constraints. Written for customers.

System Requirements

A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

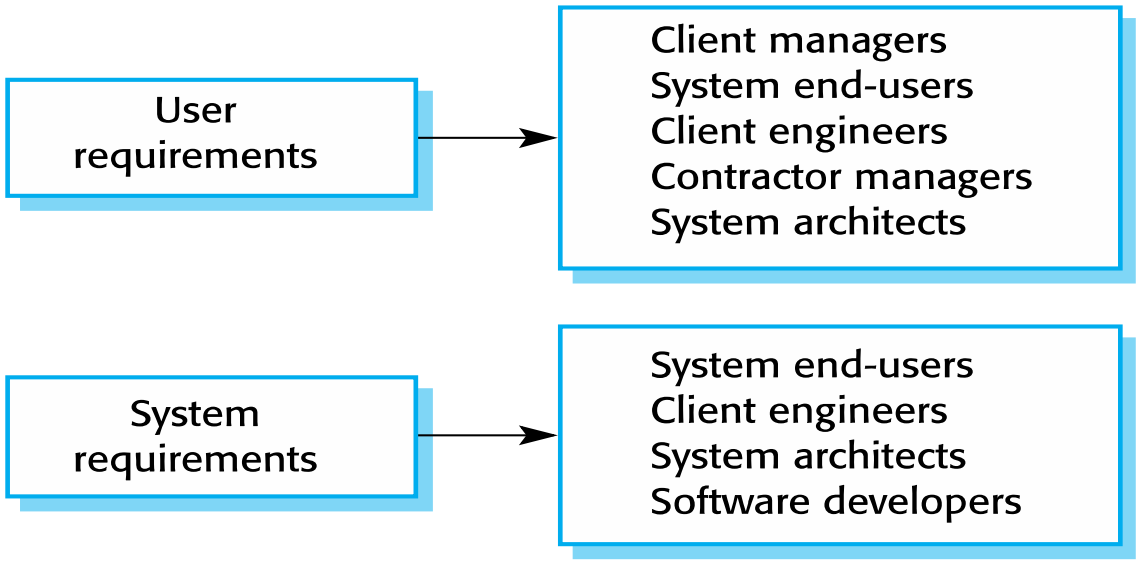


Figure 9-3: Requirements Users (Sommerville)

Functional Requirements

Statements of services the system should provide, how the system should react to inputs and how the system should behave in different situations.

May state what the system should not do.

Non-Functional Requirements

Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.

They often apply to the system rather than individual features or services.

Since there are different people involved creating and reading the requirements documents, the requirements are normally split into “High-Level Requirements” and “Detailed Requirements” (Figure 9-4).

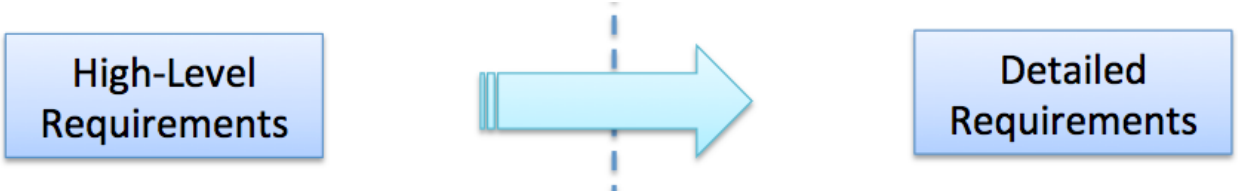


Figure 9-4: High-Level Requirements vs. Detailed Requirements

High-level requirements are for “business” people, while detailed requirements are for developers, etc.

9.1 User Requirements

User requirements are statements in natural language plus diagrams of the services the system provides and its operational constraints. User requirements are written for customers.

9.2 System Requirements

System requirements are setting out detailed descriptions of the system’s functions, services and operational constraints. They define what should be implemented may be part of a contract between client and contractor.

9.3 Functional Requirements

Functional Requirements are:

- Describe functionality or system services.
- Depending on the type of software, expected users and the type of system where the software is used.

- Functional user requirements may be high-level statements of what the system should do.
- Functional system requirements should describe the system services in detail.

9.4 Non-Functional Requirements

Non-Functional Requirements are:

- These define system properties and constraints e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc.
- Process requirements may also be specified using an IDE, programming language or development method.
- Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless.

9.5 SRS

Software Requirements Specifications (SRS) are:

- The software requirements document is the official statement of what is required of the system developers.
- It should include both a definition of user requirements and a specification of the system requirements.
- It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.

In practice, requirements and design are inseparable. Many don't separate SRS and SDD (Software Design Document) documents but include everything in a Requirements & Design Document. Such a document could be called "Software Requirements and Design Document" (SRD).

In Figure 9-5 we see some typical contents of such a SRS/SDD document.

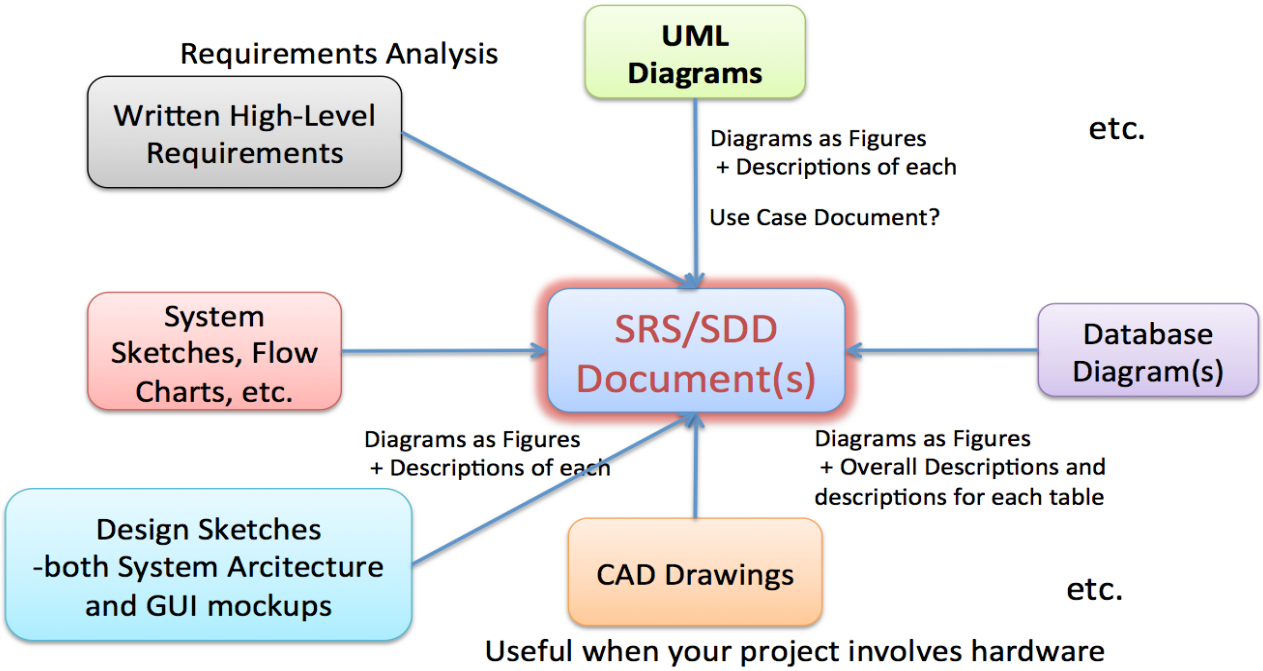


Figure 9-5: Typical SRS/SDD Contents

In Figure 9-6 we see the different users involved in the SRS document.

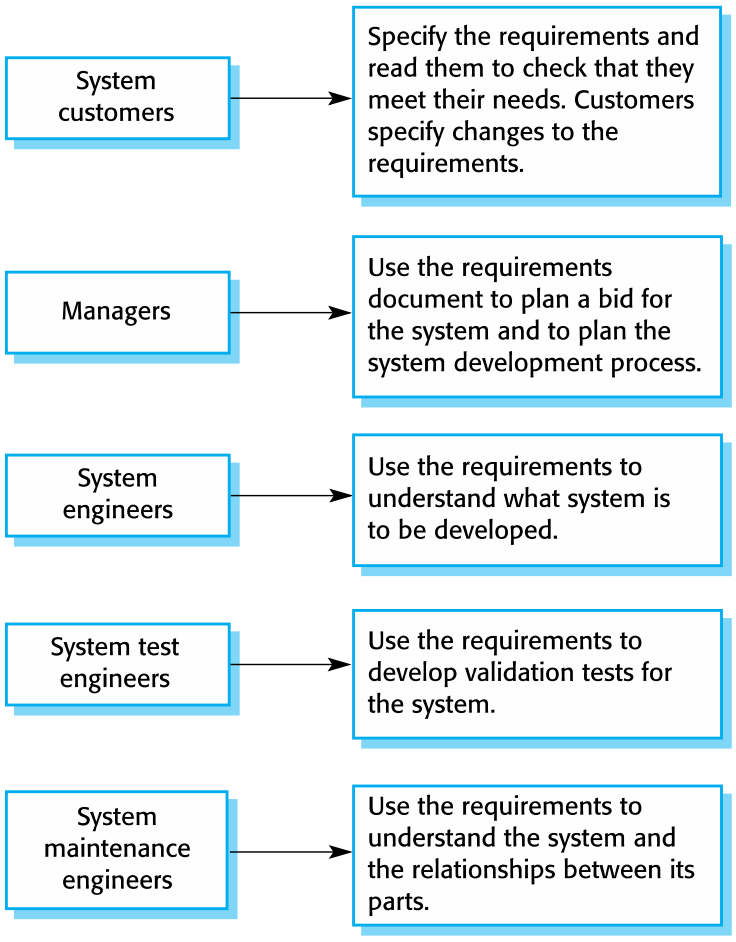


Figure 9-6: Users of SRS (Sommerville)

In Table 9-1 we see an example of what chapters that we can include in an SRS document.

Table 9-1: Example of Table of Contents for the SRS document (Sommerville)

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User Requirements Definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System Architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.
System Requirements Specification	This should describe the functional and nonfunctional requirements in more detail. If necessary, further details may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System Models	This might include graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.

System Evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed, for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

9.6 Project Estimation

Getting an overview of the total cost in a software project is important. The features and requirements need to be broken down into manageable tasks by the team. Each Tasks then needs to be estimated (Hours).

9.7 Exercises

Make sure to discuss and reflect over the following:

1. What are Software Requirements?

2. Requirements vs. Design – What is the main difference?

3. Different types of Requirements

4. What are User Requirements?

5. What are System Requirements?

6. What are Functional Requirements?

7. What are Non-Functional Requirements?

8. Give some examples of Non-Functional Requirements

9. What is SRS?

10. Mention some Requirements Analysis Problems/Challenges?

13 Testing

13.1 Introduction

Different people have come up with various definitions for Software Testing, but generally, the goal of testing is:

- To ensure that the software meets the agreed requirements and design.
- The application works as expected.
- The application doesn't contain serious bugs.
- Meets its intended use as per user expectations.
- The system must follow guidelines and regulations given by the authorities such as universal design and GDPR.

Testing can be performed on different levels and by different people. Testing is a very important part of software development. About 50% of the software development is about testing your software.

Since modern software has become very complex, testing has become a very important part of software development (see Figure 13-1).

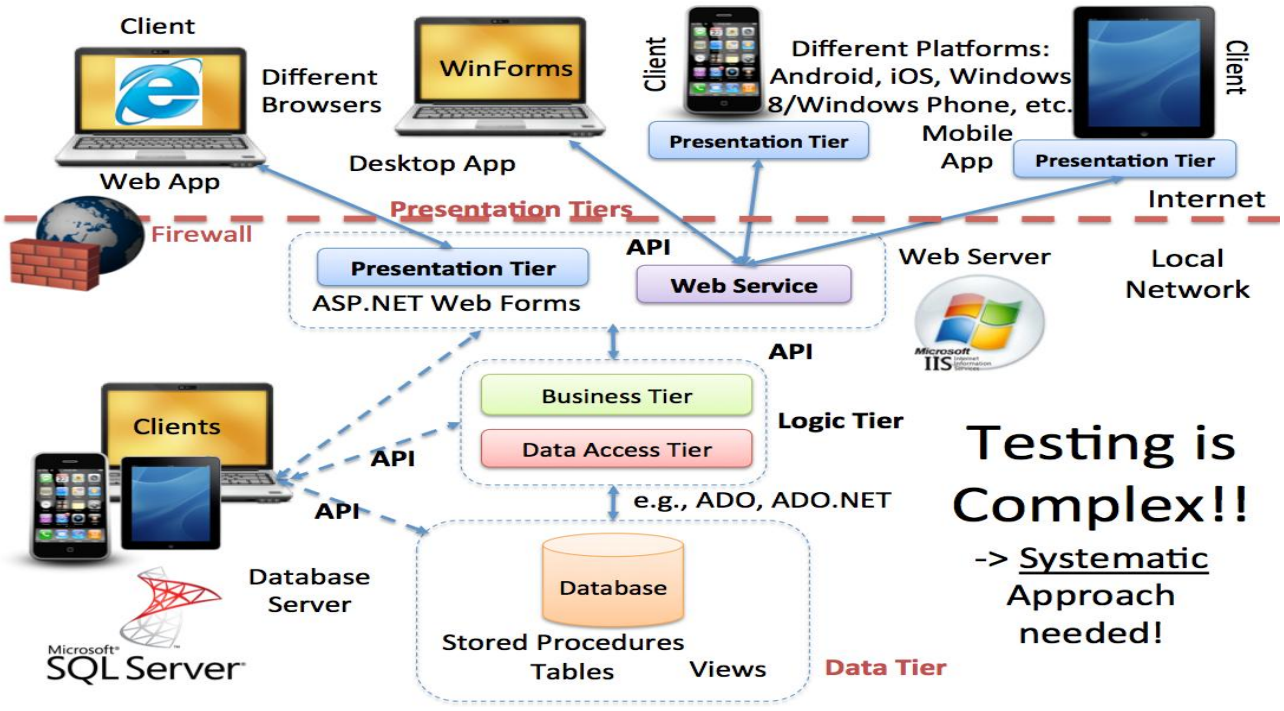


Figure 13-1: Modern Software Testing has become very complex

Since testing of advanced software systems is quite complex, we need a systematic approach to testing that involves different levels of testing (see Figure 13-2).

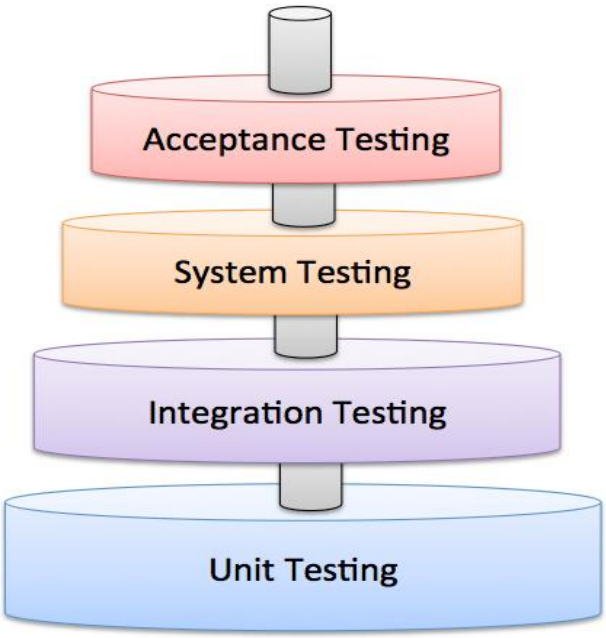


Figure 13-2: Systematic Testing

Since Software Development today involves different platforms, different devices, networks, servers, and clients, etc., it has become very complex to test it. Today we have not only ordinary Desktop Apps, but we also have Web Apps, Mobile Apps, Apps for TVs, etc.

The software we create is a layer between the user of the software and the hardware and the operating system (Figure 13-3).

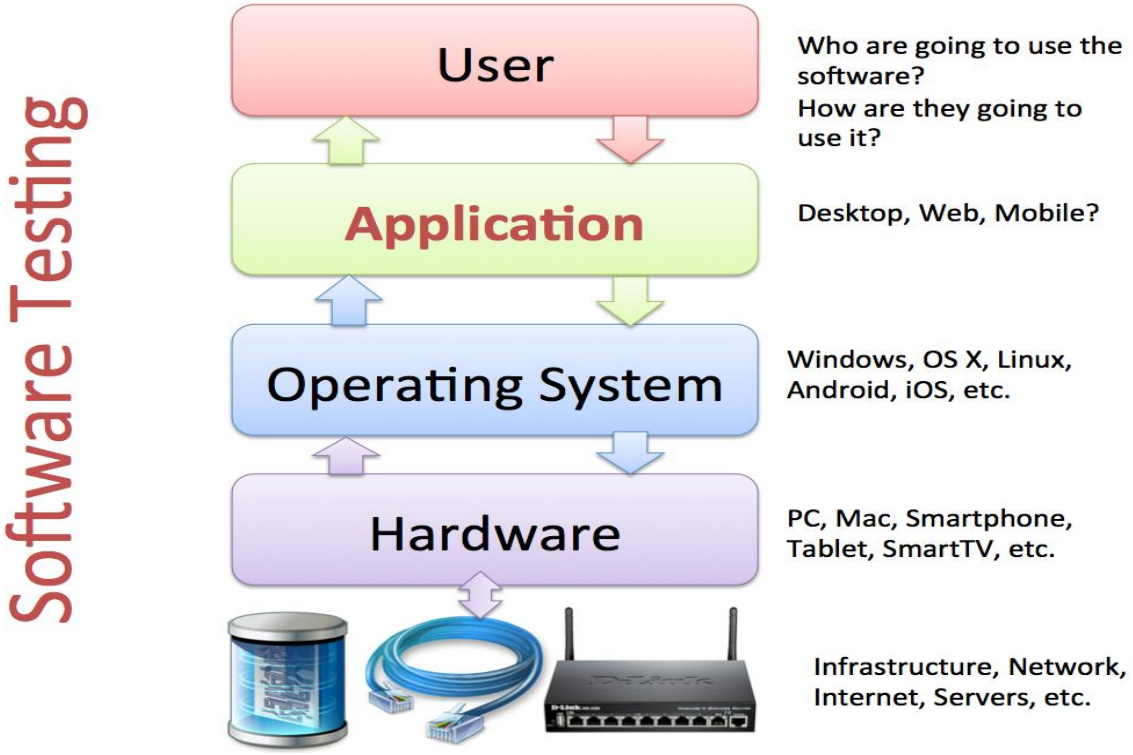


Figure 13-3: Components involved in Software Development & Testing

If we find bugs at the earlier stage, the cost to fix this will be less and thus it will reduce the overall cost of the application (Figure 13-4).

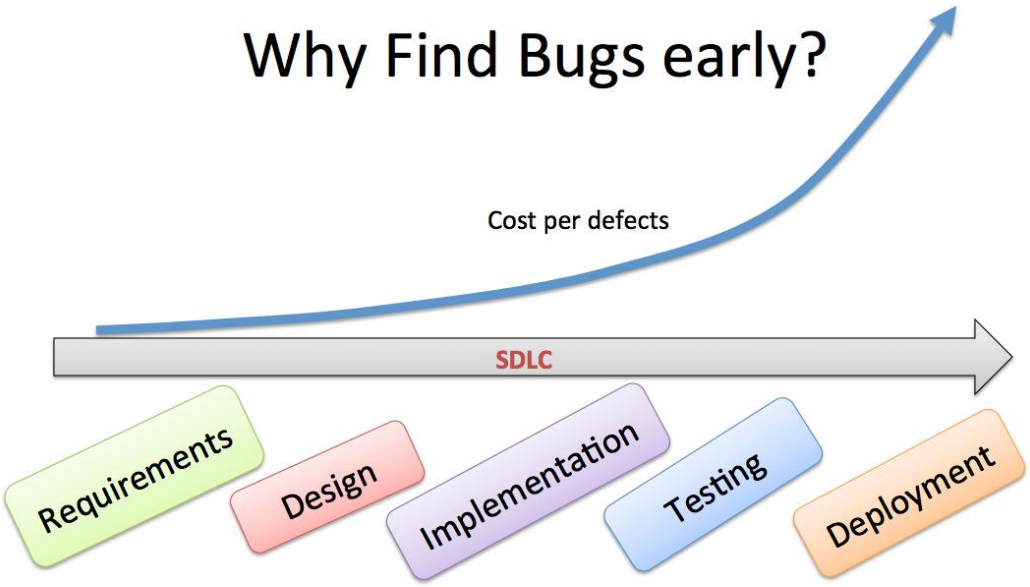


Figure 13-4: Find Bugs at an early stage

Figure 13-5 illustrates the necessary steps involved in testing.

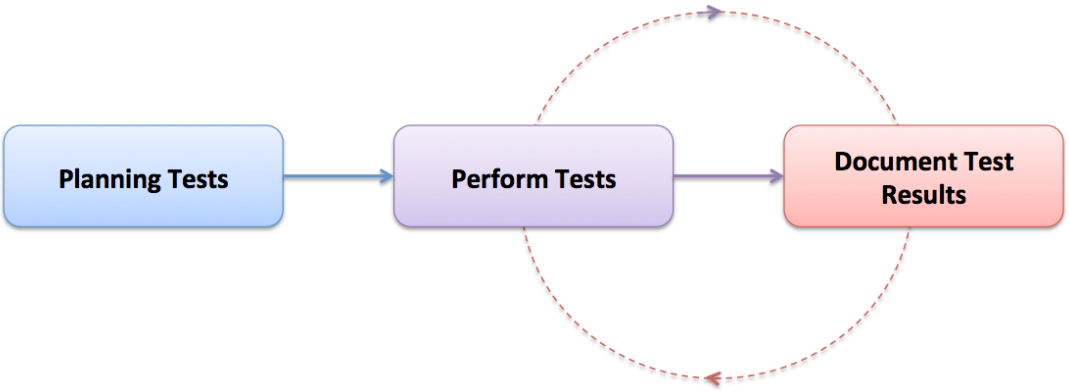


Figure 13-5: Software Testing

Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use. When you test software, you execute a program using artificial data. You check for the presence of errors NOT their absence.

Testing is part of a more general verification and validation process, which also includes static validation techniques.

What is the purpose of Testing?

The main purpose of testing is as follows:

- To demonstrate to the developer and the customer that the software meets its requirements.
    - For custom software, this means that there should be at least one test for every requirement in the requirements document.
    - For generic software products, it means that there should be tests for all the system features, plus combinations of these features, that will be incorporated in the product release.
- To discover situations in which the behavior of the software is incorrect, undesirable or does not conform to its specification.
    - This means undesirable system behavior such as system crashes, unwanted interactions with other systems, incorrect computations, and data corruption.

The primary purpose of testing is to detect software failures so that defects may be discovered and corrected.

If we summarize why we do Testing:

- Finding Bugs in the Software before it is released to the Customer
- Finding unwanted system behaviors
- Verify/Validate that the Software works as expected (according to the Specifications)
- Find bugs as soon as possible!

It is commonly believed that the earlier a defect is found the cheaper it is to fix it.

There are different steps involved in the software testing process.

The steps are as follows:

- Design Test Cases
- Prepare Test Data
- Run the Software with the necessary Test Data
- Compare the results with the Test Cases

The final output of this process is a Test Report.

Basically, we do the following: Planning the Test, then we execute the Tests, finally we document the Test results.

Documents used in testing and documents created in the test process:

- **SRS** – Software Requirements Specifications: A document stating what at application must accomplish. The documents are the basis for the test plan, etc.
- **SDD** – Software Design Document: A document describing the design of a software application. The documents are the basis for the test plan, etc.
- **STP** - Software Test Plan: Documentation stating what parts of an application will be tested, and the schedule of when the testing is to be performed
- **STD** - Software Test Documentation: Introduction, Test Plan, Test Design, Test Cases, Test procedures, Test Log, …, Summary

We have the following stages in testing:

1. **Development testing**, where the system is tested during development to discover bugs and defects. Development testing includes all testing activities that are carried out by the team developing the system.
2. **Release testing**, where a separate testing team test a complete version of the system before it is released to users.
3. **User testing**, where users or potential users of a system test the system in their own environment.

Development testing: Development testing is the responsibility of the software development team. A separate team should be responsible for testing a system before it is released to customers.

Release testing: Release testing is the process of testing a release of a system that is intended for use outside of the development team.

The primary goal of the release testing process is to convince the supplier of the system that it is good enough for use. Release testing is usually a black box testing process where tests are only derived from the system specification.

User testing: We have different types of user testing:

- **Alpha** testing - Users of the software work with the development team to test the software at the developer's site.
- **Beta** testing - A release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.
- **Acceptance** testing - Customers test a system to decide whether it is ready to be accepted by the system developers and deployed in the customer environment. Primarily for custom systems.

13.1.1 Test Levels

In Figure 13-6 we see different test levels.

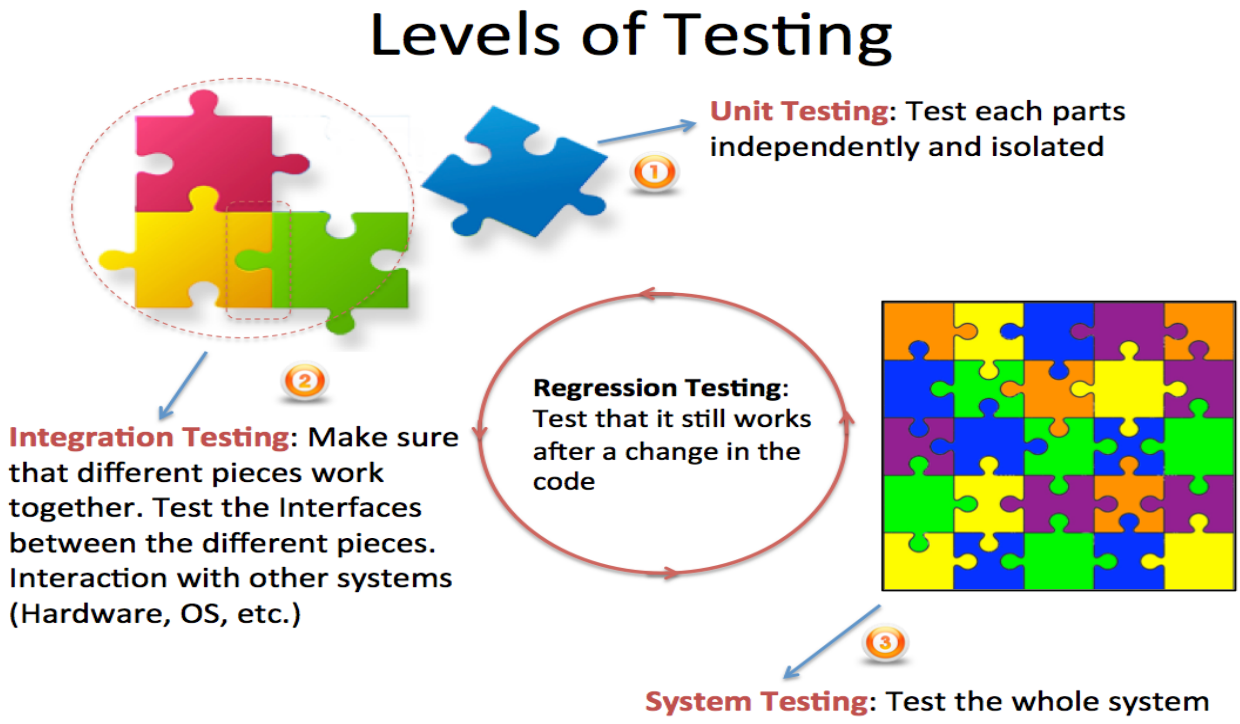


Figure 13-6: Levels of Testing

Short overview of the different Test levels in Figure 13-6 (more details later):

- **Unit Tests** are written by the Developers as part of the Programming. Each part is developed, and Unit tested separately (Every Class and Method in the code)
- **Regression testing** is testing the system to check that changes have not "broken" previously working code. Both Manually & Automatically (Re-run Unit Tests)

- **Integration testing** means the system is put together and tested to make sure everything works together.
- **System or validation testing** is Black-box Tests that validate the entire system against its requirements, i.e., checking that a software system meets the specifications
- **Acceptance Testing**: The Customer needs to test and approve the software before he can take it into use. We have 2 types: FAT (Factory Acceptance Testing) and SAT (Site Acceptance Testing).

13.1.2 Bug Tracking

A software bug is an error, flaw, failure, or fault in a computer program or system that produces an incorrect or unexpected result, or causes it to behave in unintended ways

They found a bug (a moth) inside a computer in 1947 that made the program not behave as expected. This was the “first” real bug.

13.1.3 Software versioning

Software versioning is used to separate different versions of the same software, both before it has been released and for subsequent releases. See example in Figure 13-7.

Before the software is released:

- Alpha Release(s)
- Beta Release(s)
- RC - Release Candidate(s)
- RTM – Release To Manufacturing

Maintenance (after the software is released):

- Patches (small fixes)
- SP - Service Packs
  (lots of small fixes and patches bundled together)
- …
- Start Planning next release

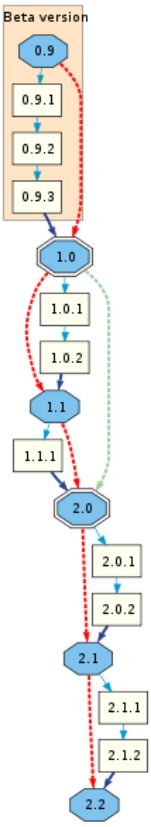


Figure 13-7: Software versioning

Software testing should be performed during the whole Software Development Life Cycle (SDLC) as shown in Figure 13-8.

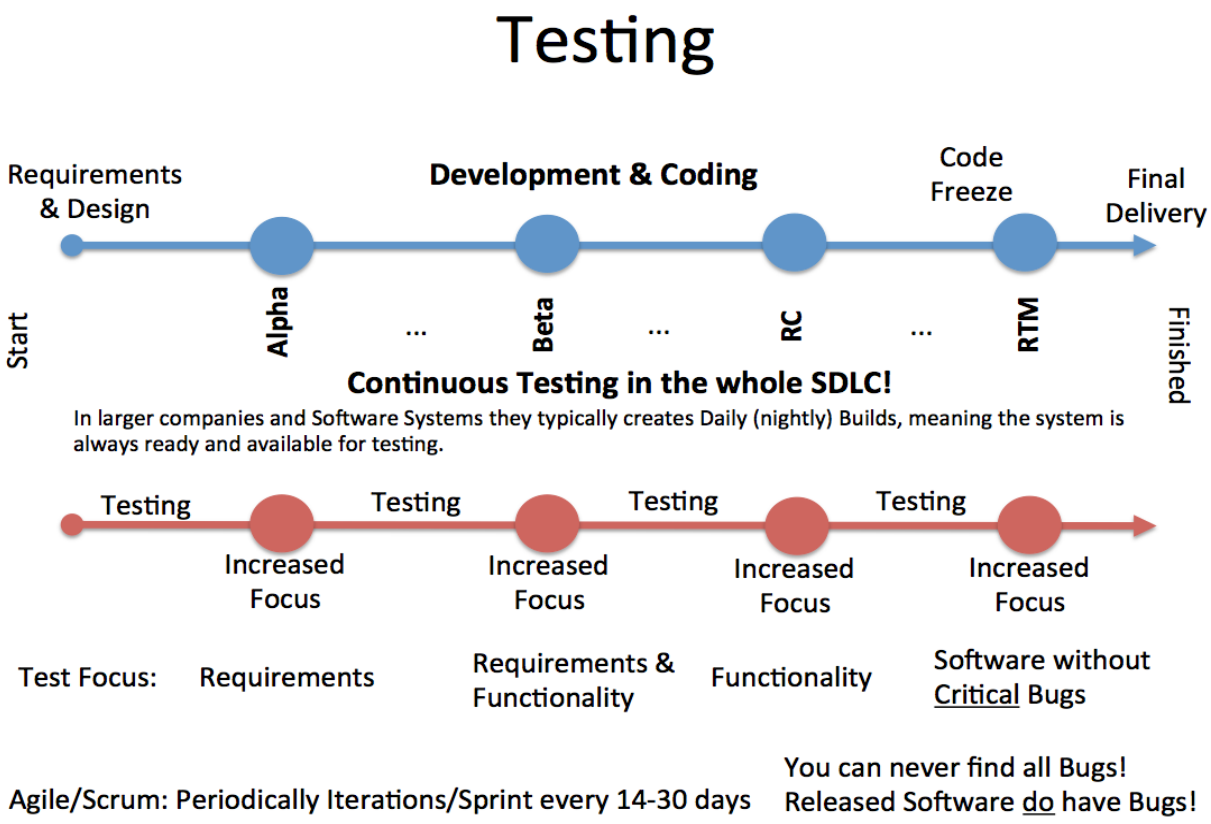


Figure 13-8: Testing during the Software Development Life Cycle (SDLC)

Sooner or later, you have to say enough is enough and release version 1.0 (see Figure 13-9).

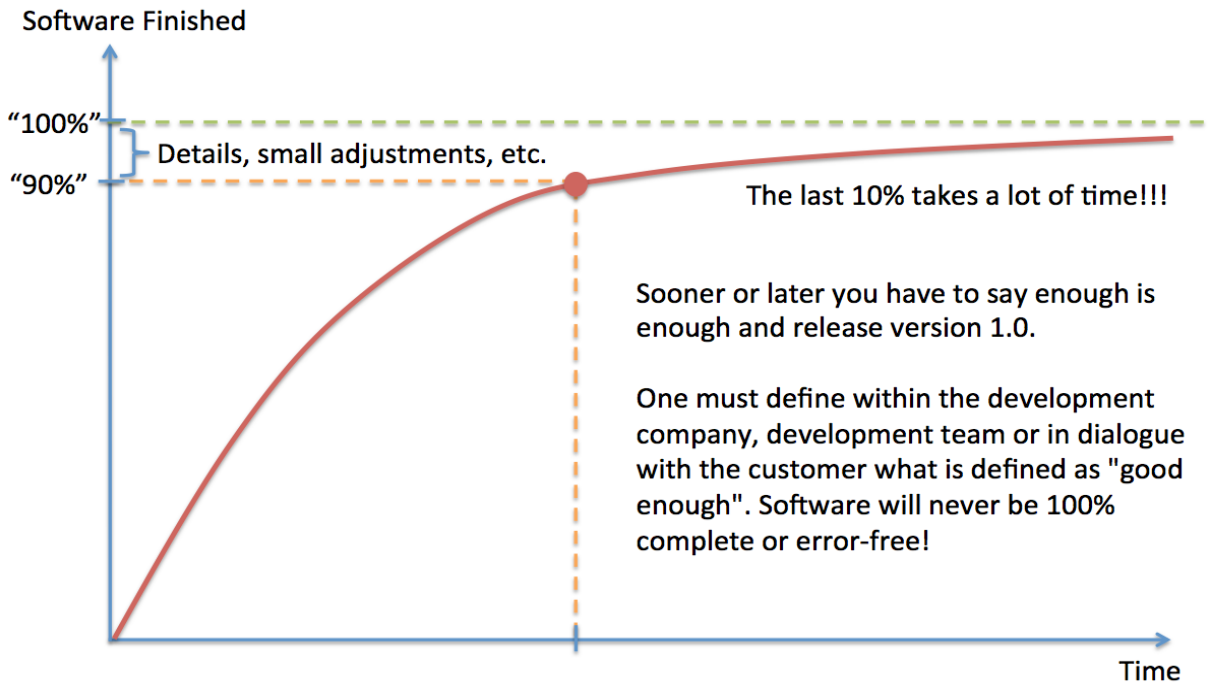


Figure 13-9: When is the Software Finished?

One must define within the development company, development team or in dialogue with the customer what is defined as "good enough".

Software will never be 100% complete or error-free (see Figure 13-10)!

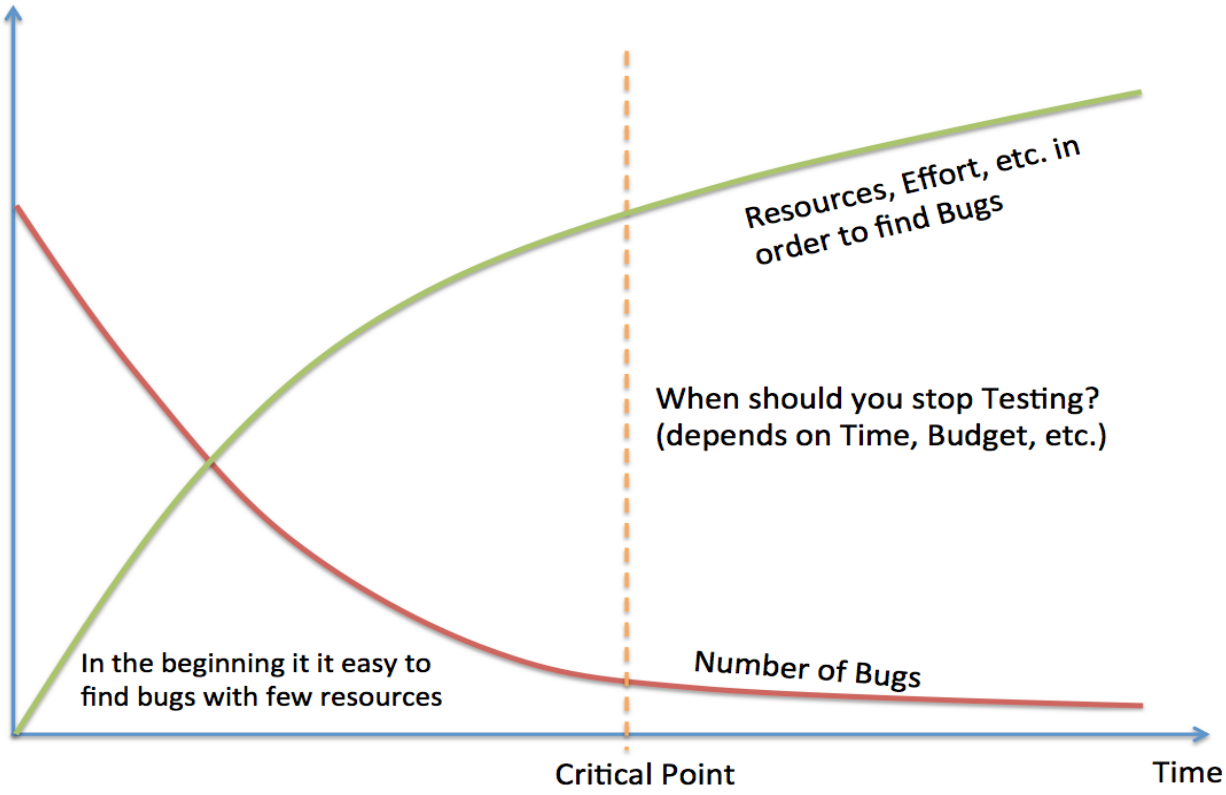


Figure 13-10: When are you Finished with Testing?

13.2 Test Categories

We can divide testing into 2 different categories, which is:

- Black-box Testing
- White-box Testing

13.2.1 Black-box Testing

Black-box testing is a method of software testing that examines the functionality of an application (what the software does) without going inside the internal structure (White-box Testing).

You need no knowledge of how the system is created. Black-box testing can be done by a person who only knows what the software is supposed to do. You may compare to driving a car – you don't need to know how it is built to test it.

13.2.2 White-box Testing

In White-box Testing you need to have knowledge of how (Design and Implementation) the system is built. White-box Testing is also called “Glass-box testing”.

In Figure 13-11 we see how White-box testing works.

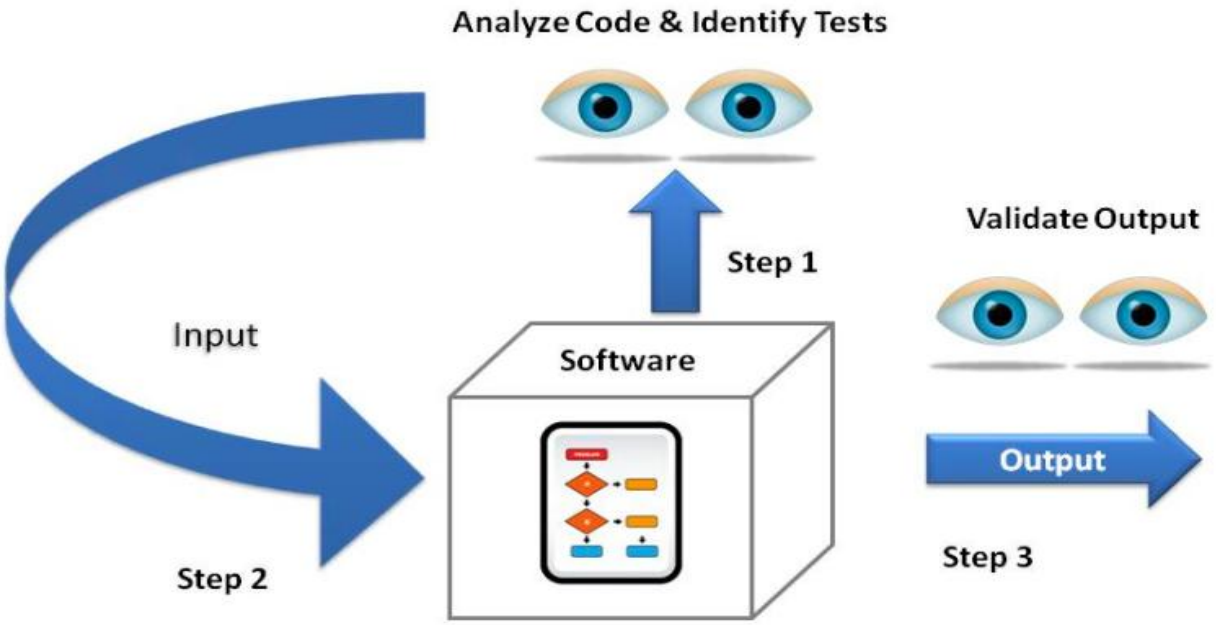


Figure 13-11: White-box Testing

13.3 Test Levels

As mentioned earlier, we have different Levels of Testing (see Figure 13-12).

- Unit Testing
- Regression Testing
- Integration Testing
- System Testing
- Acceptance Testing

These are explained in more detail below.

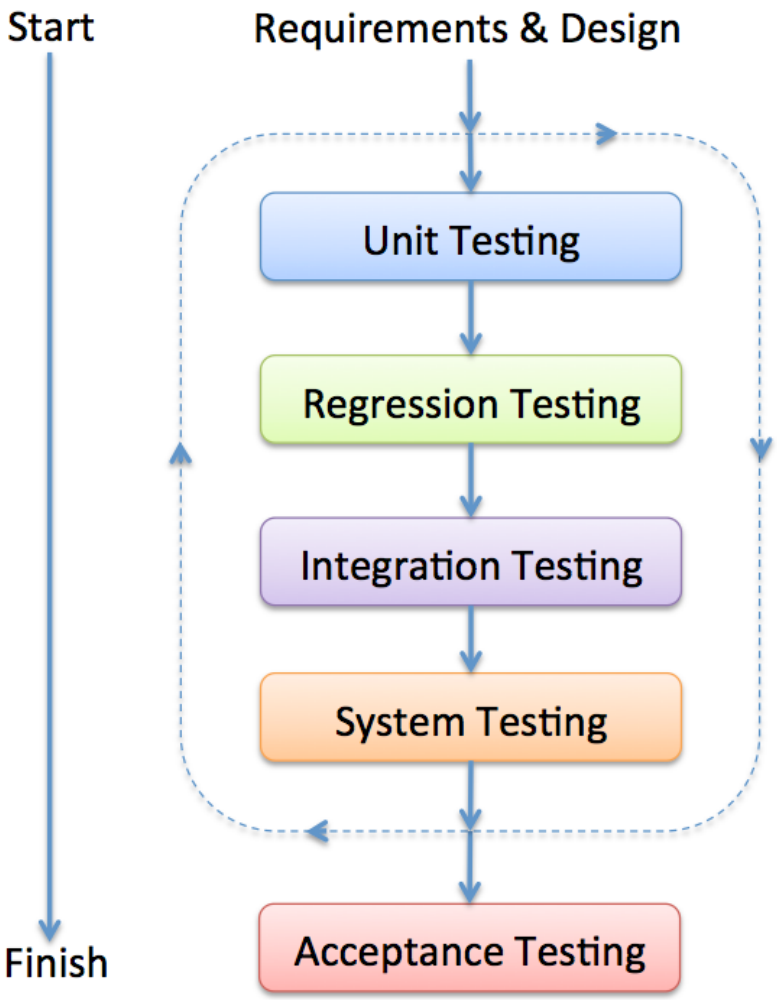


Figure 13-12: Software Test Levels

Short explanations of these Test Levels:

- **Unit Tests** are written by the Developers as part of the Programming. Each part is developed, and Unit tested separately (Every Class and Method in the code)
- **Regression testing** is testing the system to check that changes have not "broken" previously working code. Both Manually & Automatically (Re-run Unit Tests)
- **Integration testing** means the system is put together and tested to make sure everything works together.
- **System testing** is typically Black-box Tests that validate the entire system against its requirements, i.e., Checking that a software system meets the specifications
- **Acceptance Testing**: The Customer needs to test and approve the software before he can take it into use. FAT/SAT.

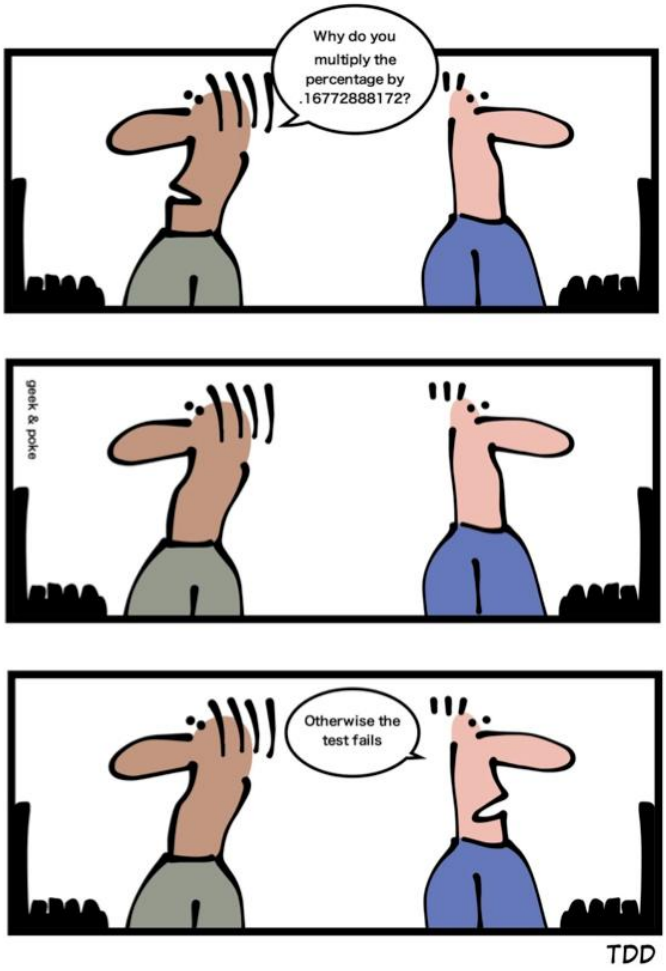
13.3.1 Unit Testing

Unit Testing (or *component testing*) refers to tests that verify the functionality of a specific section of code, usually at the function level. In an object-oriented environment, this is usually at the class and methods level.

Unit Testing in Visual Studio: https://youtu.be/QIfNViZkqEc

Unit Tests are written by the developers as part of the programming. They are automatically executed by the system, e.g., Visual Studio and Azure DevOps have built-in functionality for Unit Testing.

Sometimes the Unit Tests are written before you start programming, so-called Test-Driven Development (TDD).



[http://geek-and-poke.com]

Since Unit testing is part of the development process, so-called Unit Tests Framework are usually integrated with the IDE.

Unit Tests Frameworks:

- **Visual Studio Unit Test Framework**. Unit Tests are built into Visual Studio (no additional installation needed)
- **JUnit** (Java)
    - JUnit is a unit testing framework for the Java programming language.
- **NUnit** (.NET)
    - NUnit is an open-source unit testing framework for Microsoft .NET. It serves the same purpose as JUnit does in the Java world
- **PHPUnit** (PHP)
- LabVIEW Unit Test Framework Toolkit
- etc.

Unit Testing in Visual Studio:

Visual Studio has built-in features for Unit Testing. In the Solution Explorer you just add a "Test Project" as part of your code (see Figure 13-13).

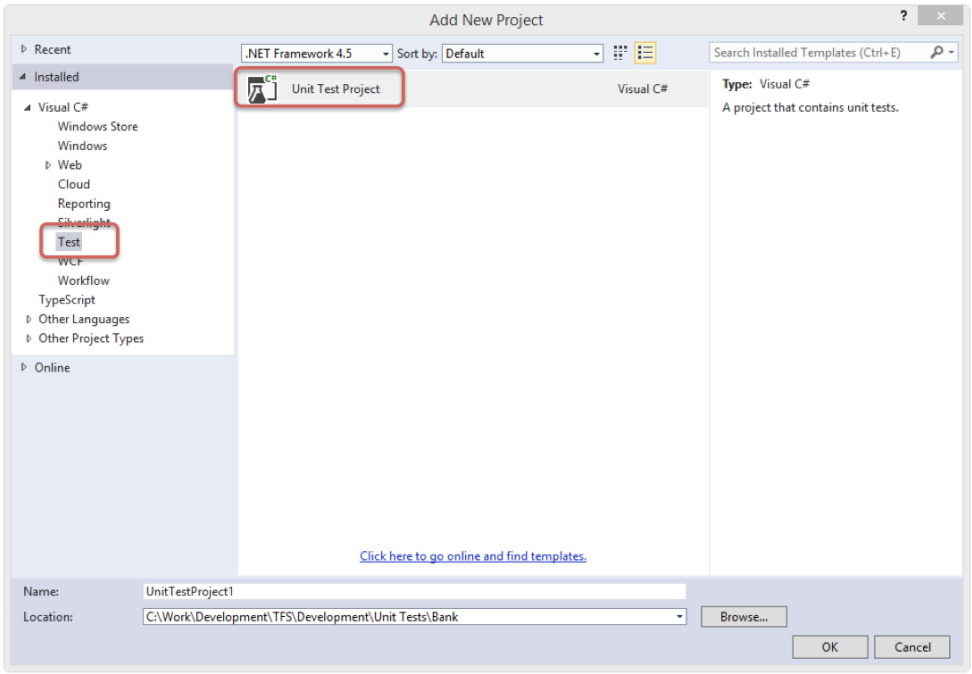


Figure 13-13: Unit Test Project in Visual Studio

In Figure 13-14 we see an example of how you create Unit Tests in Visual Studio and C#.

For Test classes, you need to use [TestClass] and for Test Methods you need to use [TestMethod]. You also need to add a reference to the code under test (select “Add Reference” in the Solution Explorer and include “using <namespace>”) in your code.

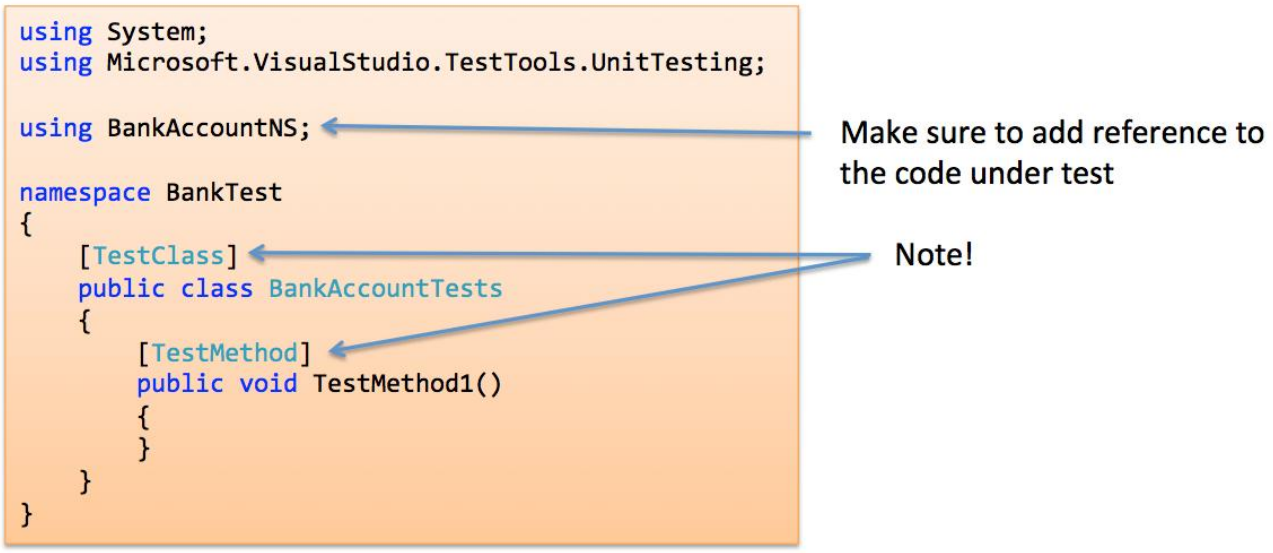


Figure 13-14: Unit Test Principle in Visual Studio and C#

The basic concept in Unit Testing is to compare the results when running the Methods with some Input Data (“Actual”) with some Known Results (“Expected”).

Example:

```
Assert.AreEqual(expected, actual, 0.001,”Test failed because...");
```

Unit Tests – Best Practice:

Here you see some basic best practice rules regarding unit tests:

- A Unit Test must only do one thing
- Unit Test must run independently
- Unit Tests must not be depending on the environment
- Test Functionality rather than implementation
- Test public behavior: private behavior relates to implementation details
- Avoid testing UI components
- Unit Tests must be easy to read and understand
- Create rules that make sure you need to run Unit Tests (and they need to pass) before you can Check-in your code in the Source Code Control System

13.3.2 Regression Testing

Regression testing focuses on finding defects after a major code change has occurred. Specifically, it seeks to uncover software regressions, or old bugs that have come back.

1. Regression testing is testing the system to check that changes have not “broken” previously working code.
2. In a manual testing process, regression testing is expensive but, with automated testing, it is simple and straightforward. All tests are rerun every time a change is made to the program.
3. Tests must run “successfully” before the change is committed.

13.3.3 Integration Testing

Integration testing verifies the interfaces between components against a software design.

13.3.4 System Testing/Validation Testing

System Testing follows Integration Testing. It consists of Black-box Tests that validate the entire system against its requirements. System Testing is about checking that a software system meets specifications and that it fulfills its intended purpose. System Testing is often executed by an independent group (QA group). QA is short for Quality Assurance.

Since system tests make sure the requirements are fulfilled, they must systematically validate each requirement in the SRS (Software Requirements Specification).

13.3.5 Acceptance Testing

Customers test a system to decide whether it is ready to be accepted from the system developers and deployed in the customer environment. It is primarily for custom systems.

In Figure 13-15 we see a typical acceptance test process.

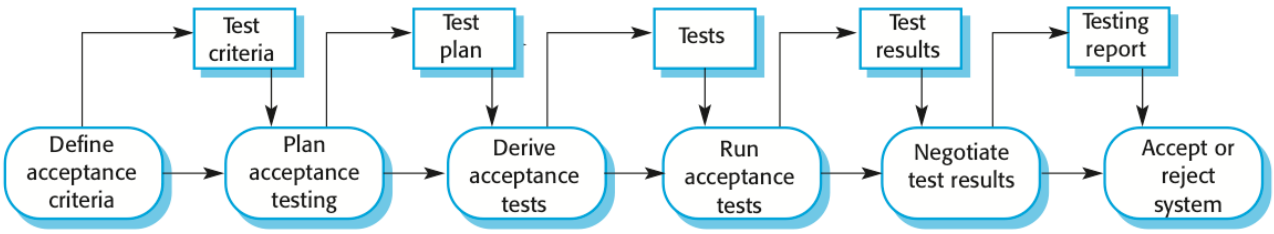


Figure 13-15: Acceptance Testing (Sommerville)

The steps are:

- Define acceptance criteria
- Plan acceptance testing
- Derive acceptance tests
- Run acceptance tests
- Negotiate test results
- Reject/accept system

We have 2 main types of Acceptance Testing:

- FAT – Factory Acceptance Testing
- SAT – Site Acceptance Testing

FAT – Factory Acceptance Testing is usually performed in the Test Environment at the software company.

SAT – Site Acceptance Testing is performed at the Customer in the actual Production Environment. This is the final step to determine if the requirements of a specification or contract are met.

If the test is accepted, the software is officially handed over to the customer.

Note! Other terms and definitions are used as well in different literature.

13.4 Universal design and GDPR

The system must follow guidelines and regulations given by the authorities such as universal design and GDPR. More about these things below.

13.4.1 Universal design

Universal design (in Norwegian: “Universell utforming” - UU) is about designing and facilitating products, environments, programs and services so that they can be used by all people, regardless of age, disability or other factors.

The goal is to create inclusive solutions that provide equal opportunities for participation and accessibility for all. That is, to create content and functionality that is accessible to everyone.

Here you will find the Norwegian Authority's guide for websites and apps:

- https://www.uutilsynet.no/veiledning/nettsteder-og-apper/711

You can use this guide to ensure that your systems meet the requirements of the WCAG standard and the authorities in the EU and Norway. More about universal design and WCAG in chapter “10.3 - Universal Design”.

In general, make sure universal design is part of your test planning and testing procedures.

13.4.2 General Data Protection Regulation (GDPR)

GDPR is short for General Data Protection Regulation. Handling data security and GDPR regulations (data protection and privacy) need to be a part of the requirements, design and the testing of the system. GDPR) is a European Union (EU) law that sets strict rules on how organizations handle personal data.

How to make sure you are GDPR compliant? Here are some important aspects:

- Collect and use personal data fairly and lawfully.
- Don't collect more data than absolutely needed.
- Allow users to access, correct, and delete their data.
- Let users manage their data.
- Keep personal data safe and secure.
- Have a privacy policy that is easy to find and read.

In general, make sure GDPR is part of your test planning and testing procedures.

13.5 Test Documentation

In Figure 13-16 we see the steps involved in the software testing process.

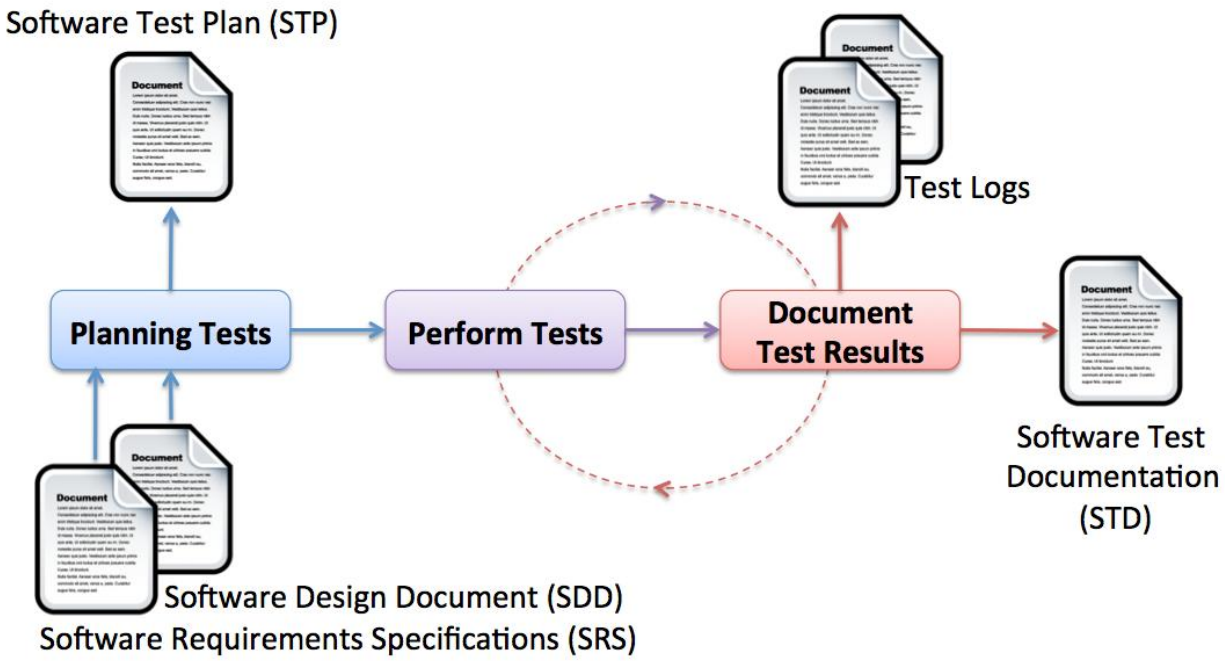


Figure 13-16: The Software Testing Process

Documents involved:

- **SRS** – Software Requirements Specifications: A document stating what at application must accomplish
- **SDD** – Software Design Document: A document describing the design of a software application
- **STP** - Software Test Plan: Documentation stating what parts of an application will be tested, and the schedule of when the testing is to be performed
- **STD** - Software Test Documentation: Introduction, Test Plan, Test Design, Test Cases, Test procedures, Test Log, …, Summary

In addition to writing different documents in your test phase, you should have a Bug Tracking System. With a Bug Tracking System, you can easily store all your bugs in a database system, set priorities, use search to find bugs, use different statistics, etc. More about Bug Tracking Systems below.

13.5.1 Test Planning and Software Test Plan (STP)

Test planning involves scheduling and estimating the system testing process, establishing process standards, and describing the tests that should be carried out. As well as helping managers allocate resources and estimate testing schedules, test plans are intended for software engineers involved in designing and carrying out system tests. They help technical staff get an overall picture of the system tests and place their own work in this context.

As well as setting out the testing schedule and procedures, the test plan defines the hardware and software resources that are required. Test plans are not static documents but evolve during the development process. Test plans change because of delays at other stages in the development process. Test planning is particularly important in large software system development. For small and medium-sized systems, a less formal test plan may be used, but there is still a need for a formal document to support the planning of the testing process.

A **Software Test Plan (STP)** document typically answers the following:

Testing should be based on Requirements & Design Documents

- What shall we test?
- How shall we test?
- Hardware/Software Requirements
- Where shall we test?
- Who shall test?
- How often shall we test (Test Schedule)?

- How shall tests be documented? It is not enough simply to run tests; the results of the tests must be systematically recorded. It must be possible to audit the testing process to check that it has been carried out correctly
- System tests: This section, which may be separate from the test plan, defines the test cases that should be applied to the system. These tests are derived from the system requirements specification.

13.6 Bug Tracking Systems

All the results from the testing need to be documented, stored, and tracked.

For this purpose, we use a so-called Bug Tracking System.


Bug Reporting and Tracking with Azure DevOps: https://youtu.be/0tlWdqWdFeQ

Here are some popular Bug Tracking Systems in use today:

- Azure DevOps
- Jira
- Bugzilla
- ClearQuest

More about Bug Tracking Systems in Chapter 23 - Bug Tracking Systems.

We will focus on Azure DevOps in this document. The bug tracking features in Azure DevOps will be discussed in another chapter.

In Azure DevOps we can add requirements, user stories, tasks, new features, bugs, etc. as so-called “Work Items” (Figure 13-17).

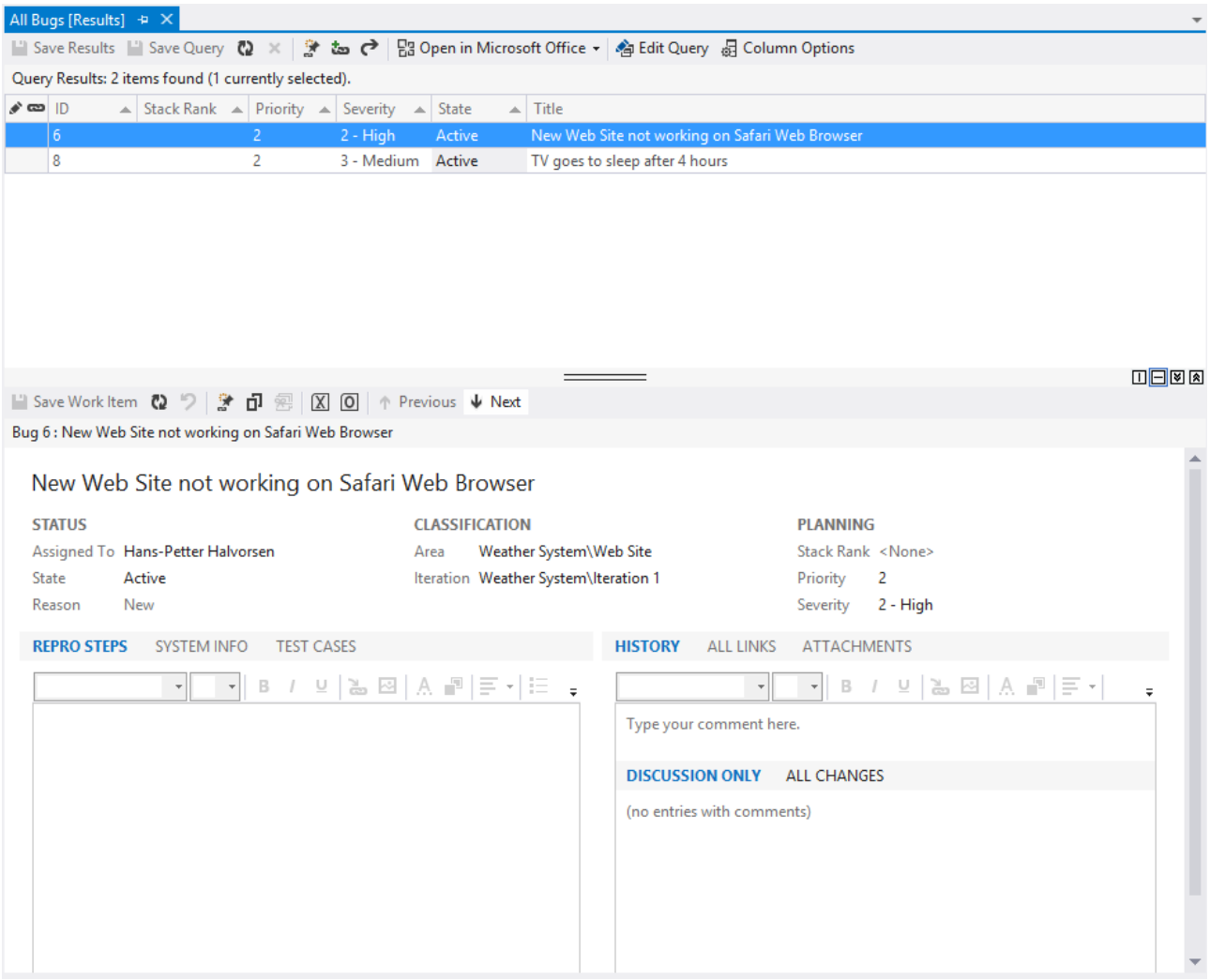


Figure 13-17: Azure DevOps – Work Items

13.7 Test Environment

A testing environment is a setup of software and hardware on which the testing team is going to perform the testing of the newly built software product.

VirtualBox - Installation of Windows 11 in a Virtual Machine:
https://youtu.be/OA9urLN4DWo

Install WinForm Desktop App in Virtual Test Environment using VirtualBox:
https://youtu.be/g7CPEVFT8AA

 Install ASP.NET Core Web App in Virtual Test Environment using VirtualBox: https://youtu.be/7XrRd7voasI

Here are also some other resources using VMware Workstation:

 ASP.NET Core - Deploy to Virtual Test Environment: https://youtu.be/WzJIoP4WpmU

Why do we need a Test Environment? Here are some reasons:

- "It works on my PC" says the Developer
- We need a Clean Environment when testing
- On the Developers PCs, we have all kind of Software installed that the Customer don't have, e.g., Development Tools like Visual Studio, etc.
- We need to test on different Platforms and Operating Systems
- Customers may use different Web Browsers
- Deployment: Test of Installation packages
- Make the software available for Testers
- etc.

This setup consists of a physical setup which includes hardware, and logical setup that includes Server Operating system, client operating system, database server, front end running environment, browser (if web application), IIS (version on server side) or any other software components required to run this software product.

This testing setup is to be built on both the ends – i.e., the server and client.

To set up such environments, virtualization is the answer.

More about virtualization below.

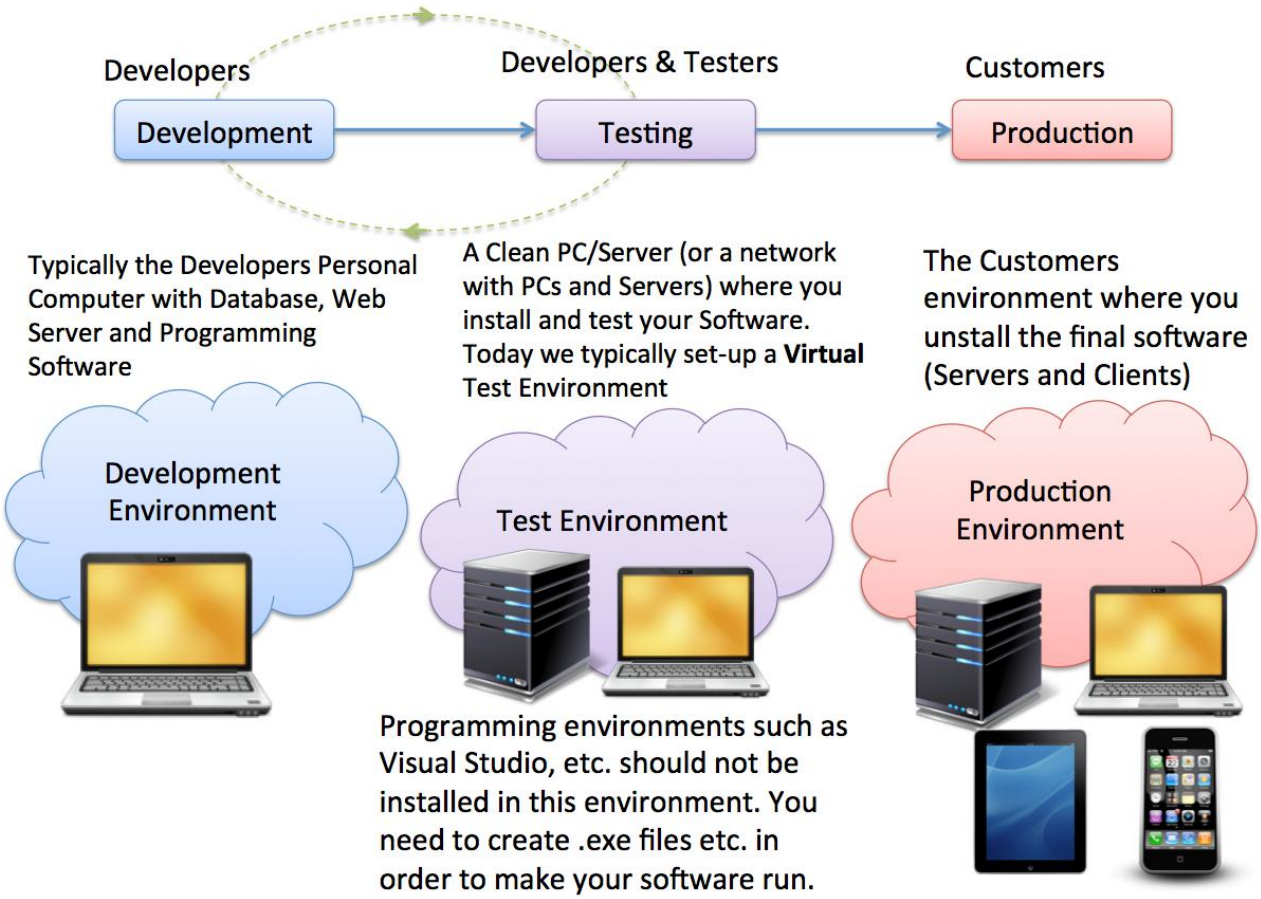
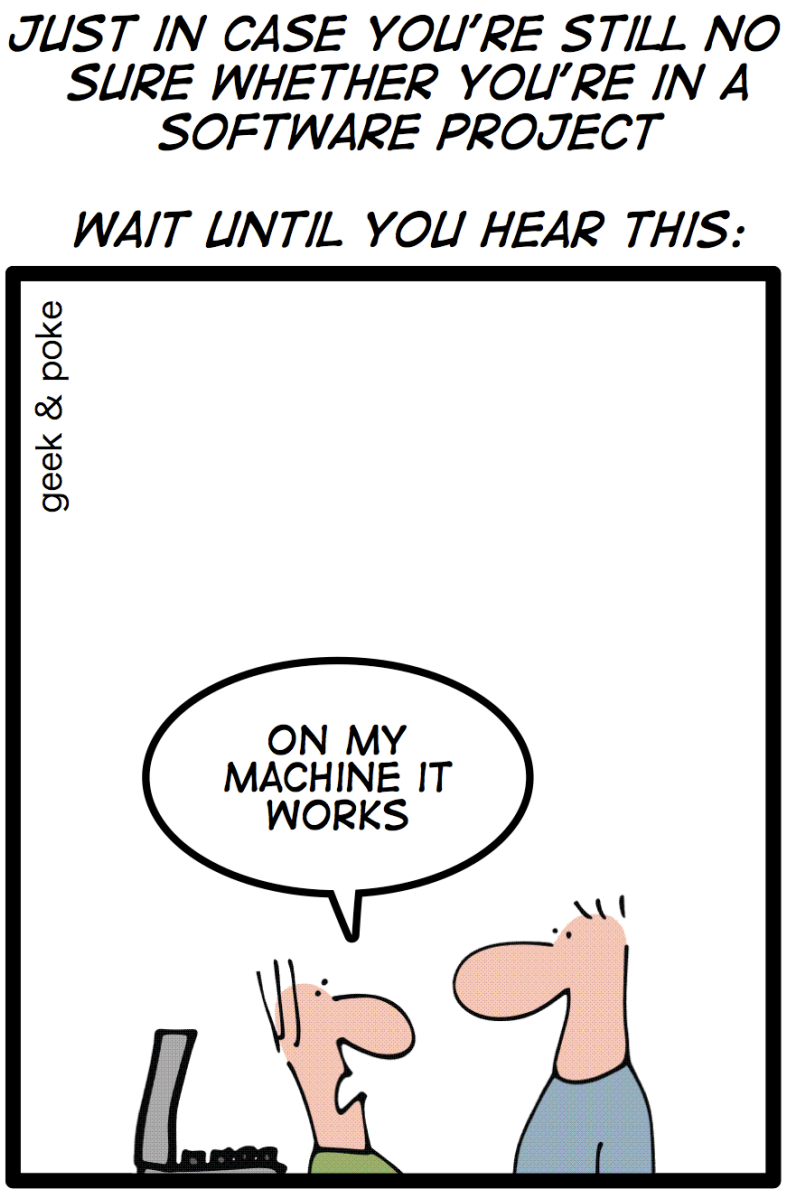


Figure 13-18: Development, Test and Production Environment



[http://geek-and-poke.com]

13.7.1 Virtualization

To create test environments easily, virtualization is the answer. There exists lots of different virtualization solutions on the market today.

Here are some examples:

- VirtualBox
- VMware Workstation
- VMware vSphere
- Microsoft Hyper-V
- VMware Fusion (Mac)

- Parallels Desktop (Mac)
- etc.


VirtualBox - Installation of Windows 11 in a Virtual Machine:
https://youtu.be/OA9urLN4DWo


Install WinForm Desktop App in Virtual Test Environment using VirtualBox:
https://youtu.be/g7CPEVFT8AA


Install ASP.NET Core Web App in Virtual Test Environment using VirtualBox:
https://youtu.be/7XrRd7voasI

VirtualBox is a free virtualization tool. VMware Workstation is another virtualization tool.

With such tools you can create so-called Virtual Machines (VM) where you can install and run all kinds of software.

In this way, you can easily test your software without destroying your own computer and you can easily test it in different operating systems, etc.

13.8 Terms used in Testing

Here we will discuss some terms used in software testing not covered earlier.

13.8.1 Bugs

A software bug is an error, flaw, failure, defect, or fault in a computer program or system that produces an incorrect or unexpected result or causes it to behave in unintended ways.

They found a bug (a moth) inside a computer in 1947 that made the program not behave as expected. This was the “first” real bug.

13.8.2 Debugging

Debugging is about different techniques for finding and fixing bugs (errors that make your code not work as expected) in your code. It is difficult to write code without errors (bugs), but e.g., Visual Studio and other tools have powerful Debugging functionality (breakpoints, etc.). The Compiler will also find syntax errors, etc.

For more “advanced” bugs other methods are required (Unit Testing, Integration Testing, Regression Testing, Acceptance Testing, etc.). The focus here will be on these methods, while Debugging is something you learned in Programming courses.

13.8.3 Code Coverage

Code coverage is a measure used in software testing. It describes the degree to which the source code of a program has been tested.

Example:

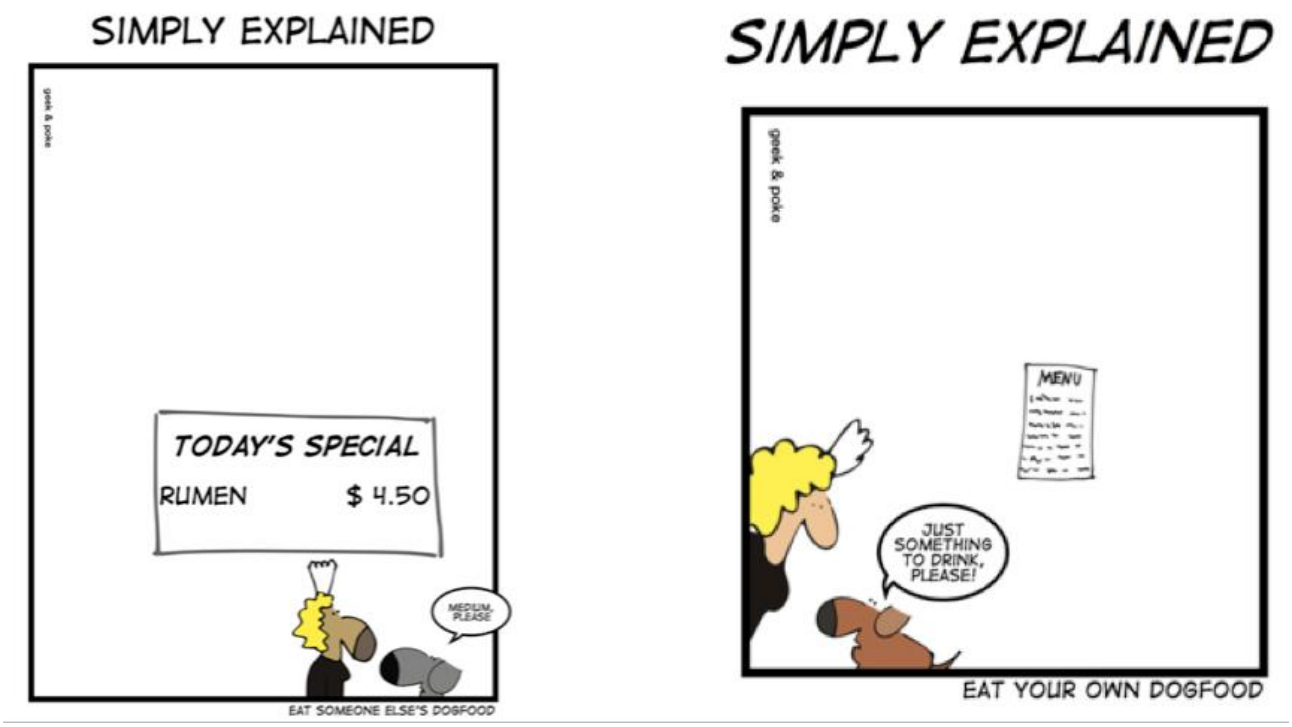
```
int foo (int x, int y)
{
    int z = 0;
    if ((x>0) && (y>0))
    {
        z = x;
    }
    return z;
}
```

When we test this function, it depends on the input arguments which parts of the code will be executed. Unit Tests should be written to cover all parts of the code.

13.8.4 Eat your own Dog food

“Eating your own dog food”, also called “dogfooding”, is a slang term used to reference a scenario in which a company (usually, a computer software company) uses its own product to demonstrate the quality and capabilities of the product.

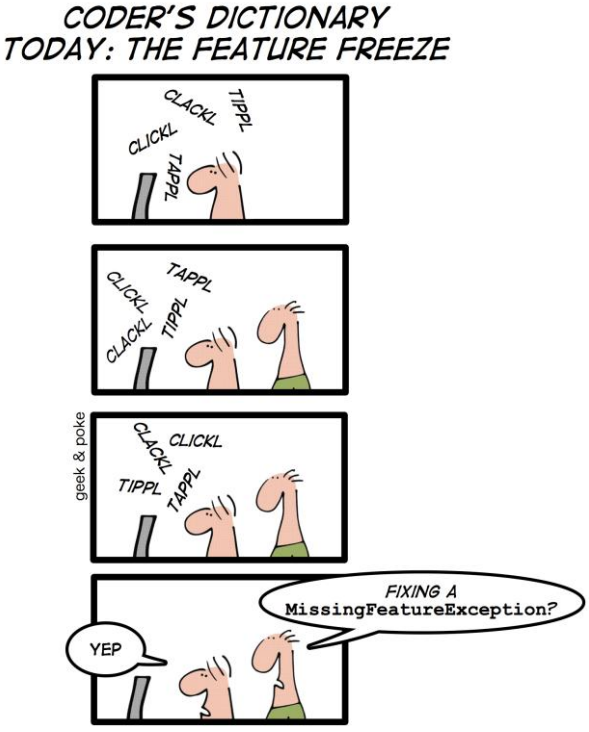
Example: Microsoft uses Windows PCs and Visual Studio to create their software.



[http://geek-and-poke.com]

13.8.5 Code/Feature Freeze

The Developer cannot add new features to the software, only fix bugs. When it is very close to release, they cannot fix bugs either.



[http://geek-and-poke.com]

13.8.6 Test-Driven Development (TDD)

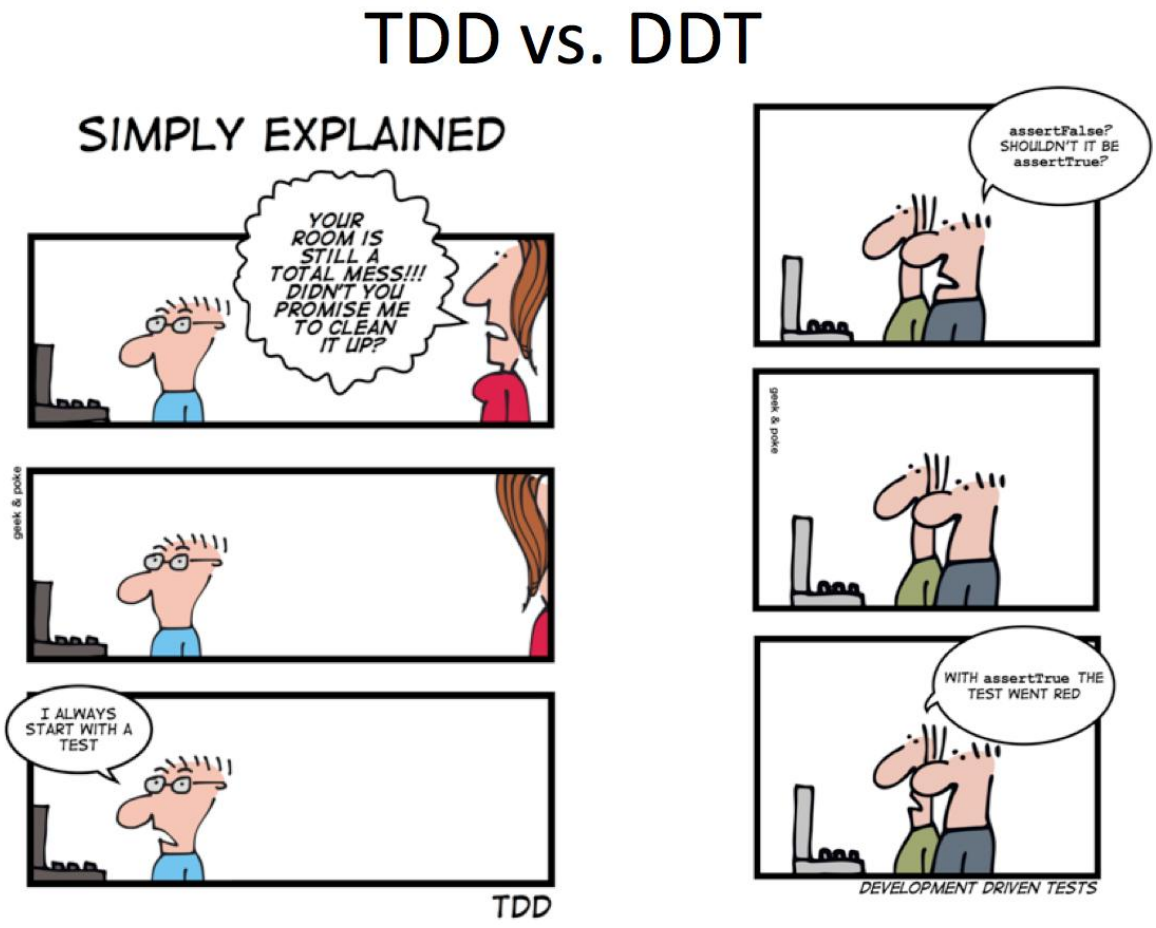
In TDD coding and testing are done in parallel. The tests are normally written before the code. TDD was introduced as part of eXtreme Programming (XP).

13.8.7 Development-Driven Testing (DDT)

DDT is all about giving more responsibility to developers specifically, and the development process in general. It works especially well when using test cases as requirements, and having the developers write these test cases. But it's not DDT unless those tests are written near the end of the process, when the code is checked in, and the developers figure they're done.

The advantages of Development-Driven Testing are many. Instead of tests driving the development, it's developers driving the tests, so you get just a few tests, and they almost always all pass. The project team can deliver on time for a change, with zero bugs found in every iteration. This makes management happy, and isn't that really the ultimate barometer of success? Also, velocity is increased dramatically when using this process.

Development-driven testing makes all sense in the world for those who practice Agile.



[http://geek-and-poke.com]

13.9 The 7 Principles of Testing

The 7 Principles of Testing are as follows:

1. **Testing shows the presence of Bugs**: Software Testing reduces the probability of undiscovered defects remaining in the software but even if no defects are found, it is not a proof of correctness.
2. **Exhaustive Testing is impossible**: Testing everything is impossible! Instead, we need optimal amount of testing based on the risk assessment of the application.
3. **Early Testing**: Testing should start as early as possible in the Software Development Life Cycle (SDLC)
4. **Defect Clustering**: A small number of modules contain most of the defects/bugs detected.
5. **The Pesticide Paradox**: If the same tests are repeated, eventually the same test cases will no longer find new bugs
6. **Testing is Context dependent**: This means that the way you test a e-commerce site will be different from the way you test a commercial off the shelf application
7. **Absence of Error is a Fallacy**: Finding and fixing defects does not help if the system build is unusable and does not fulfill the users' needs and requirements

For more information about these 7 principles of testing, see the following:

7 Principles of Software Testing with Examples: http://www.guru99.com/software-testing-seven-principles.html

13.10 Testing Summary

Figure 13-19 gives an overview of different Test Categories, Test Levels and Test Methods.

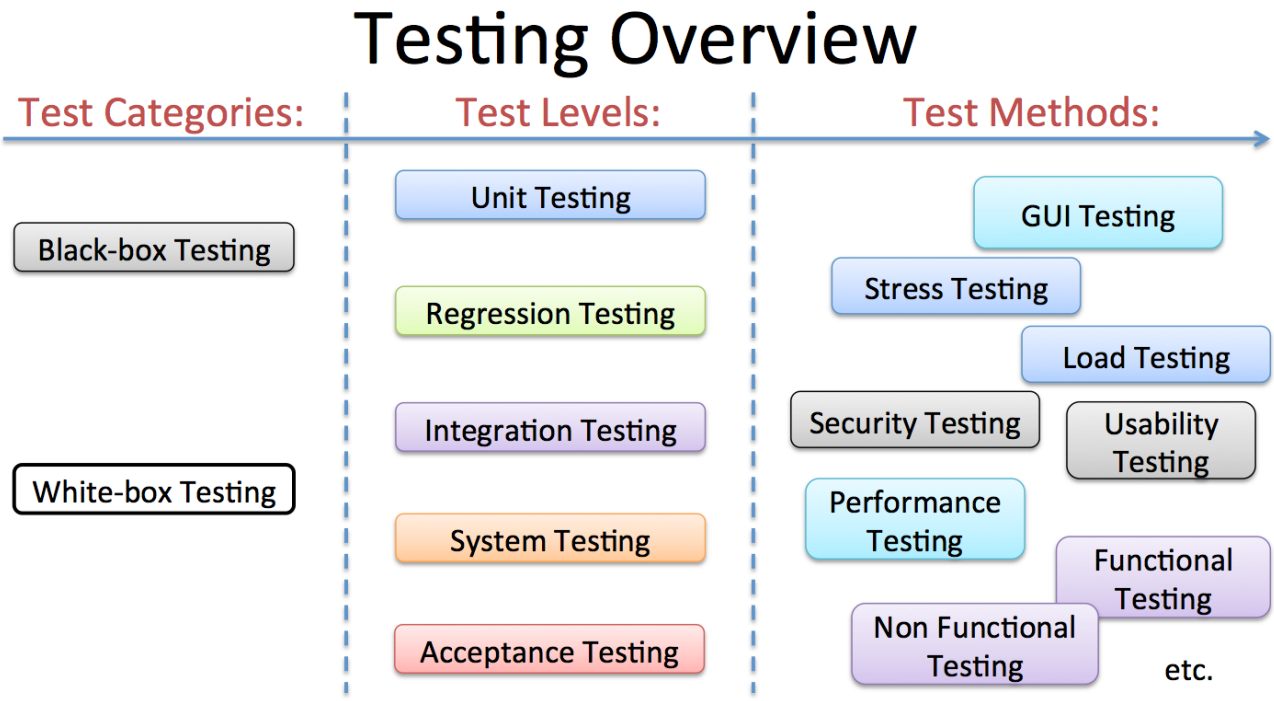


Figure 13-19: Test Categories, Test Levels and Test methods

13.11 Exercises

Make sure to discuss and reflect over the following:

1. Why do we need to test the software?
2. List different Test methods
3. We have 2 main categories of testing. Explain.
4. Explain the difference between a “Bug” and a “Feature”
5. What is Code/Feature Freeze?
6. What is “Dogfooding”?
7. What is a Code Review?
8. What Explain TDD and DDT?

9. What is Unit Testing?

10. What is the difference between Functional and Non-Functional Testing?

19 Software Architecture

When creating software, we use different architecture depending on the platform and the purpose with the software.

In this document, we will focus on client-server, 3-tier architecture and creating and using Web Services and APIs.

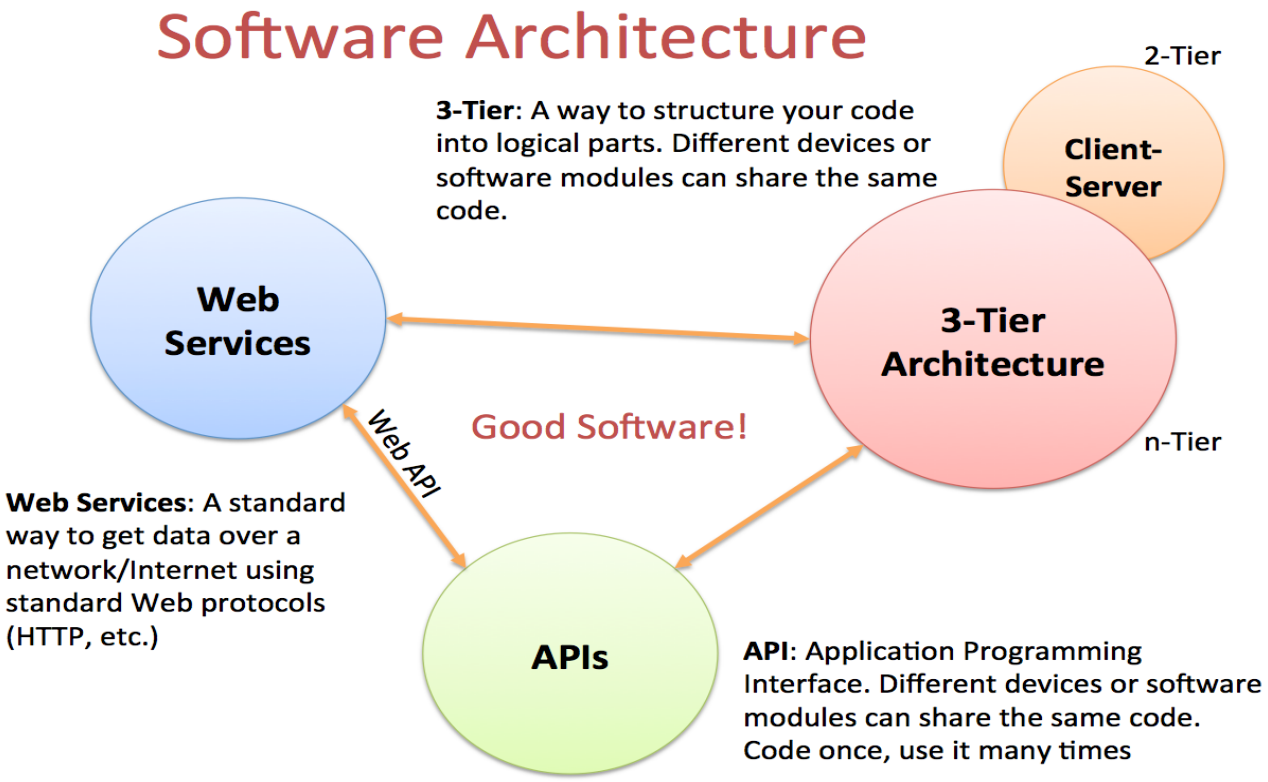


Figure 19-1: Software Architecture

In Figure 19-2 we see how a typical software application is interacting with the surrounding environments, such as the users of the software and the underlying operating system (which is also software) and hardware.

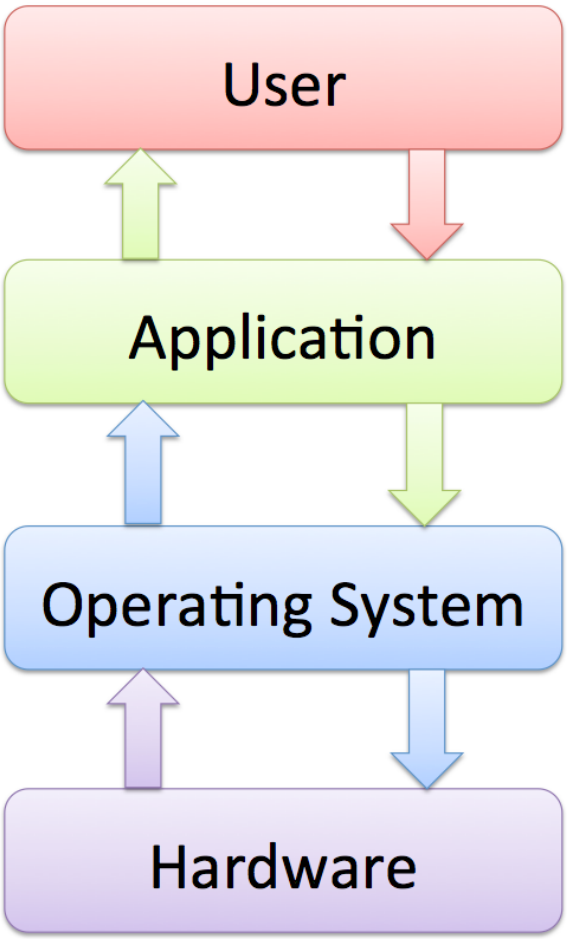


Figure 19-2. Software Interaction with the Environment

Based on the fundamental interaction between the software and the environment we have different kinds of software architecture.

Software Architecture Examples:

- Client – Server
- n-tier architecture, 3-tier architecture
- Model-View-Controller (MVC)
- Web Services
- Interfaces
- APIs
- Service-oriented architecture (SOA)
- Microservices

In Figure 19-3 we see some examples of different network and software architecture typically used in software development.

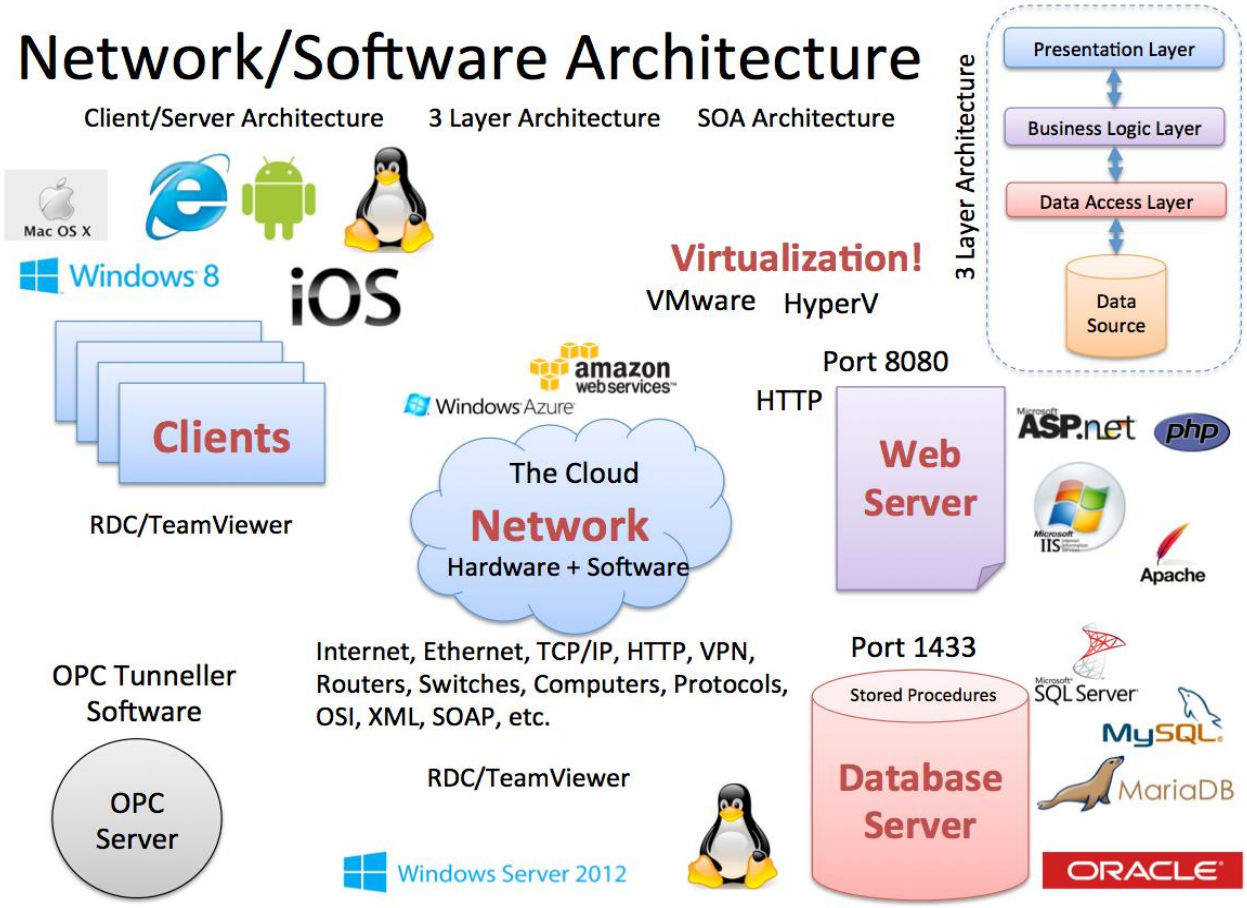


Figure 19-3: Network/Software Architecture Examples

Make sure to document your system and your architecture in your documentation. You should typically include several drawings and sketches of your architecture.

Make System Sketch in PowerPoint: https://youtu.be/9mmBXFOjV3s

19.1 API

API - Application Programming Interface. An API is a specification of how some software components should interact with each other. Typically, it is a library with functions, etc. you can use in your code.

Examples:

- Windows API
- Java API

But you can also create your own API that you use internally in the team or an API that you expose to others.

Creating APIs is good practice and makes it easy to reuse your code in other components or applications. If all the developers in the team create the same code without thinking of reusing code from others or creating code in such manners as it can't be used by others, the software project is doomed to fail.

Software Design without APIs:

Here are some pros and cons regarding APIs:

Pros:

- Fast to implement small projects.
- Agile – can serve as a starting point for API design.
- No need to consider how code interfaces with other software.
- Can be appropriate for small "dead end" projects.

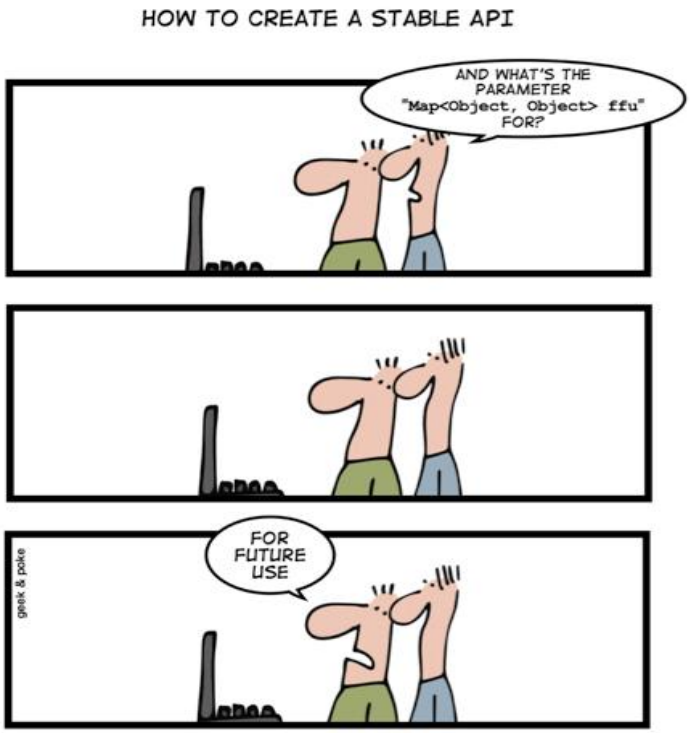
Cons:

- Inappropriate for large projects.
- Code has a limited (as opposed to general) functionality.
- Code is not reusable.
- Code is hard to maintain/modify.
- Prone to errors and bugs.

Why a Good API is hard to Design?

Here are some examples:

- Forces designer to anticipate future usage of code.
- Requirements are incomplete (may never be complete).
- Requires abstraction.
- Requires modularization.
- Requires skills in programming languages.
- Requires code rewrites – time consuming and labor intensive.



[http://geek-and-poke.com]

When an API is used in a project, it

- Allows us to focus on the project.
- Saves development time.
- Reduces errors and debugging.
- Facilitates modular design.
- Provides a consistent development platform.

API driven design requires planning and programming skills. API driven design is costly initially, but it pays in the long run. So, obviously, creating APIs is good software practice in most cases.

It is impossible to imagine how anyone would design a car today without taking advantage of existing modules or vehicle subsystems – it is the same with software!

19.2 Client-Server

Client/server describes the relationship between two computer programs in which one program, the client, makes a service request from another program, the server, which fulfills the request. Although the client/server idea can be used by programs within a single computer, it is a more

important idea in a network. In a network, the client/server model provides a convenient way to interconnect programs that are distributed efficiently across different locations.

19.3 Web Services and Web APIs

The “problem”: How do we share data between different devices in a network (see Figure 19-4)?

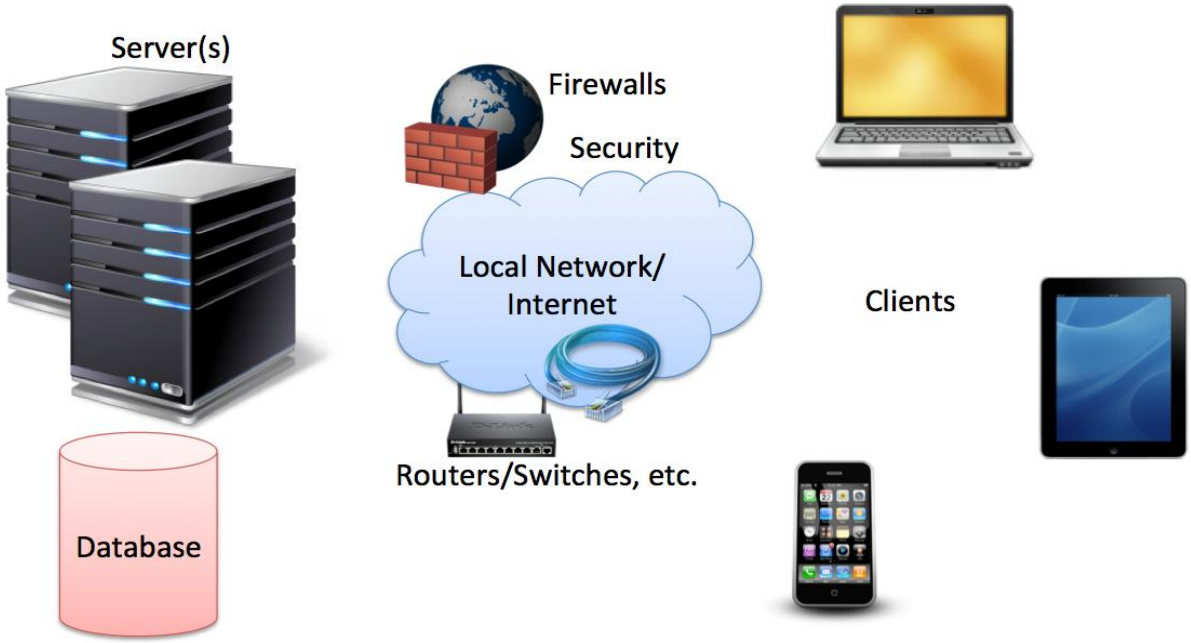


Figure 19-4: Data sharing between devices in a network

Direct connection between the database and the clients that need the data is normally not possible, due to security, compatibility issues, etc. (firewalls, hacker attacks, etc.). Direct connection in a local network (behind the firewall) is normally OK – but not over the Internet (see Figure 19-5).



Figure 19-5: Limited access to the database in a network

The solution: Web Services. Web Services use standard web protocols like HTTP, etc. HTTP is supported by all Web Browser, Servers, and many Programming Languages.

Today Web Services have been very popular. A Web service is a method of communication between two devices over the World Wide Web and makes it easy to share data over a network or the internet.

A Web Service is:

- A Web API
- A Standard defined by W3C
- Cross-platform and Platform-independent Communication
- Distributed Application Development

Web Services can be implemented and used in most Programming Languages (C#/ASP.NET, PHP, LabVIEW, Objective-C, Java, etc.)

Web services use standard web technology (web protocols) such as **HTTP**, **REST**, SOAP, **XML**, WSDL, **JSON**, etc.

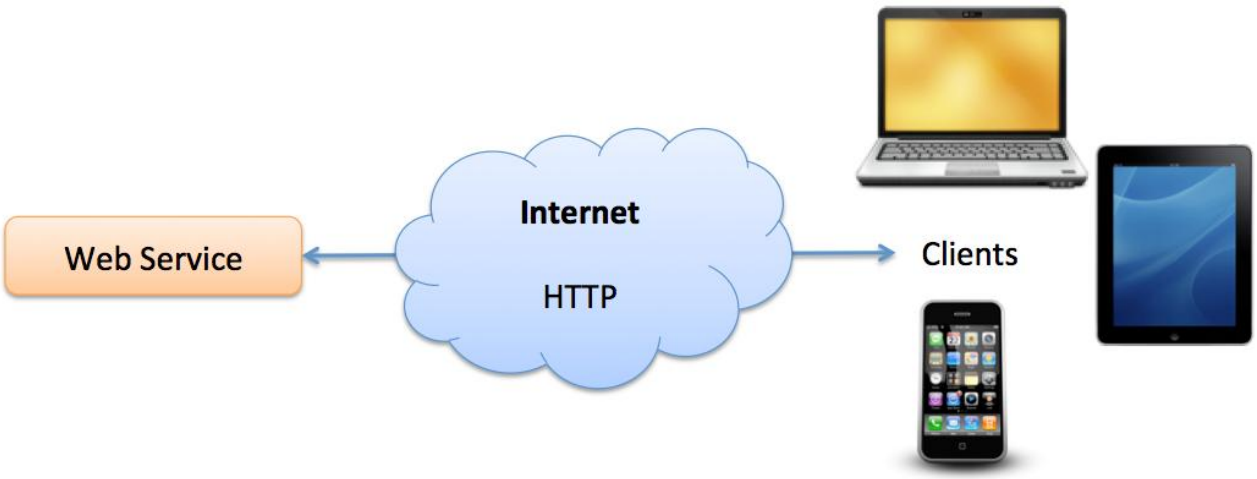


Figure 19-6: Web Services and Web APIs

Web services technology used in web services:

- **HTTP** - Hypertext Transfer Protocol
- **XML** – Extensible Markup Language
- WSDL - Web Services Description Language
- SOAP - Simple Object Access Protocol
- **REST** - Representational State Transfer

(We will not go into details)

A web service is typically deployed on a web server, similar as ordinary web pages, see Figure 19-7.

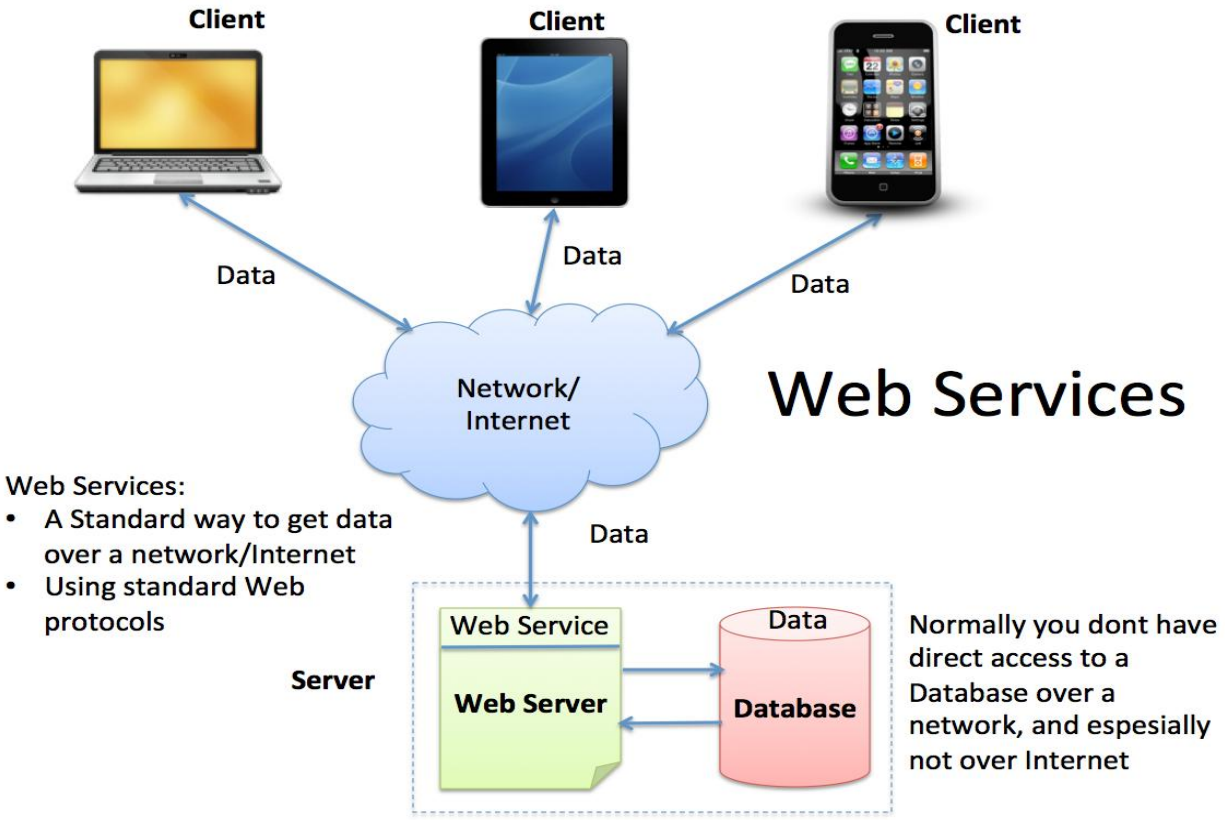


Figure 19-7: Web Service Infrastructure

We have 2 different types of web services:

- Web Services 1.0: SOAP Web Services
  - "Complex"
- Web Services 2.0: REST web services or web APIs
  - Lightweight and Flexible
  - A new and simpler version of traditional web services
  - All major web services or web APIs on the Internet today use REST

In Figure 19-8 we summarize Web Services 1.0 vs. 2.0.



Figure 19-8: Different kind of Web Services

This document only describes the basic principles of web services and web APIs.

19.3.1 SOAP Web Services

In Figure 19-9 we see the different "layers" a "SOAP Web Service" consists of.

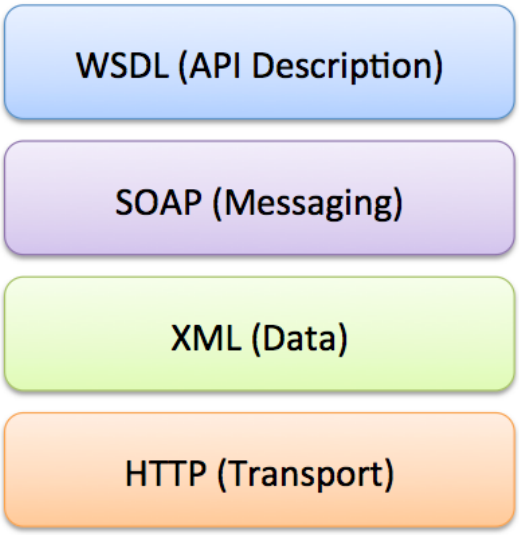


Figure 19-9: SOAP Web Services Architecture

XML:

XML stands for eXtensible Markup Language. XML is designed to transport and store data.

Below we see an XML document example.

```
<?xml version="1.0"?>
```

```
<note>
    <to>Tove</to>
    <from>Jani</from>
    <heading>Reminder</heading>
    <body>Don't forget me this weekend!</body>
</note>
```

19.3.2 REST Web Services

In Figure 19-10 we see the different “layers” a “REST Web Service” consists of.

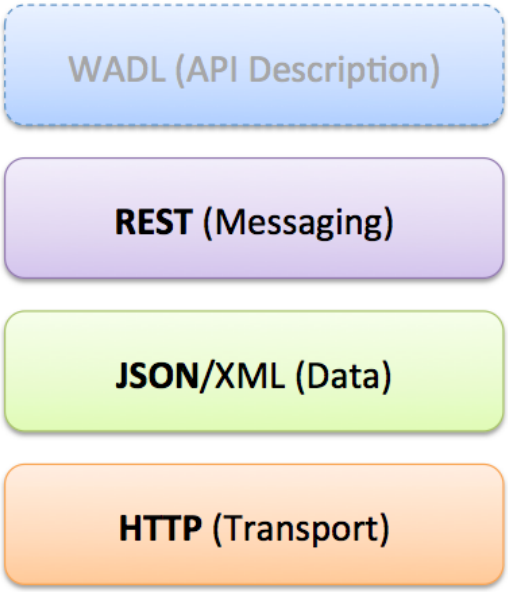


Figure 19-10: REST Web Services Architecture

19.3.3 Creating Web Services with Visual Studio

Visual Studio has powerful features for creating Web Services and Web APIs (Figure 19-11).

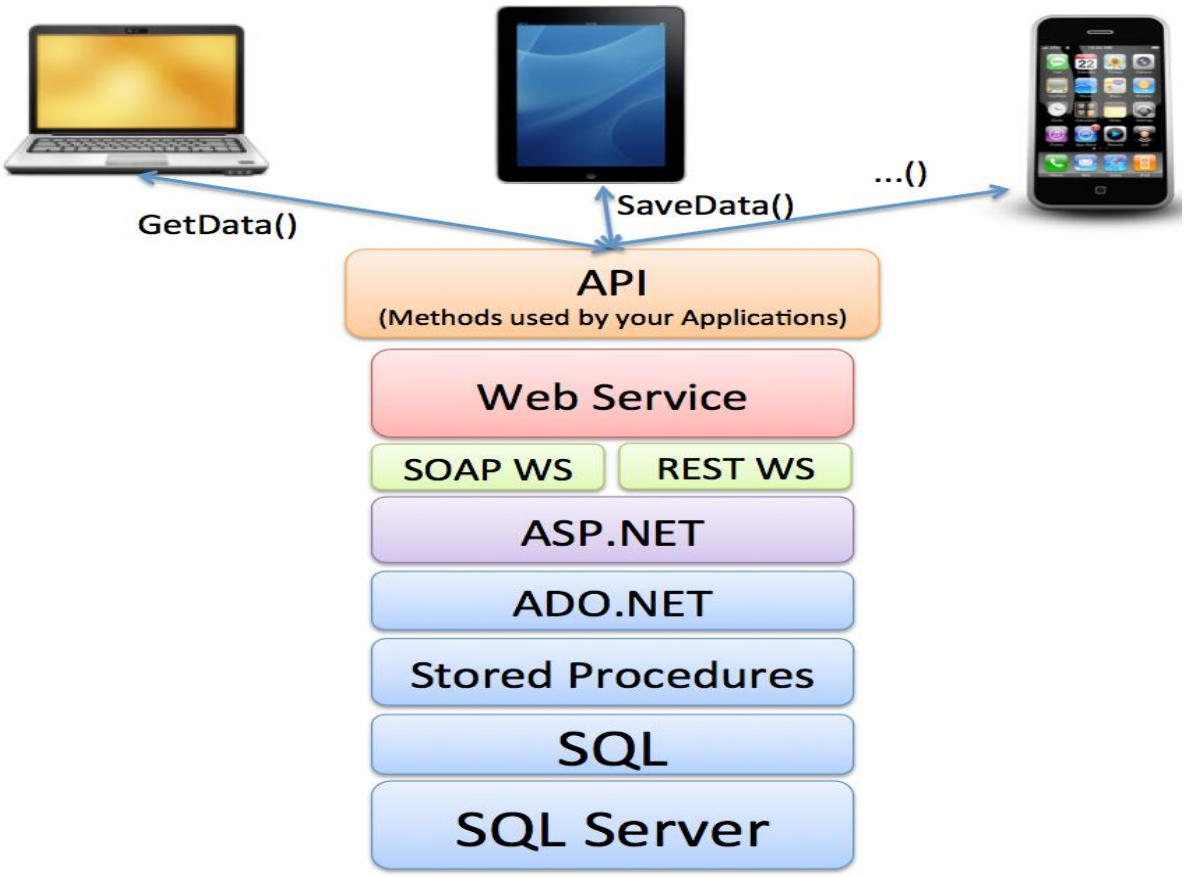


Figure 19-11: Creating Web Services using Visual Studio and ASP.NET

19.4 3-tier Architecture

In general, we have so-called n-tier architecture, but the most common version is a 3-tier architecture.

3-tier architecture is a client-server architecture in which the functional process logic, data access, computer data storage and user interface are developed and maintained as independent modules on separate platforms. Three-tier architecture is a software design pattern and a well-established software architecture. Its three tiers are the presentation tier, application tier and data tier.

The 3 tiers or layers are as follows:

- Presentation Layer
- Business/Application Layer
- Data Access Layer

These layers may be on the same computer, but normally they are distributed on different computers.

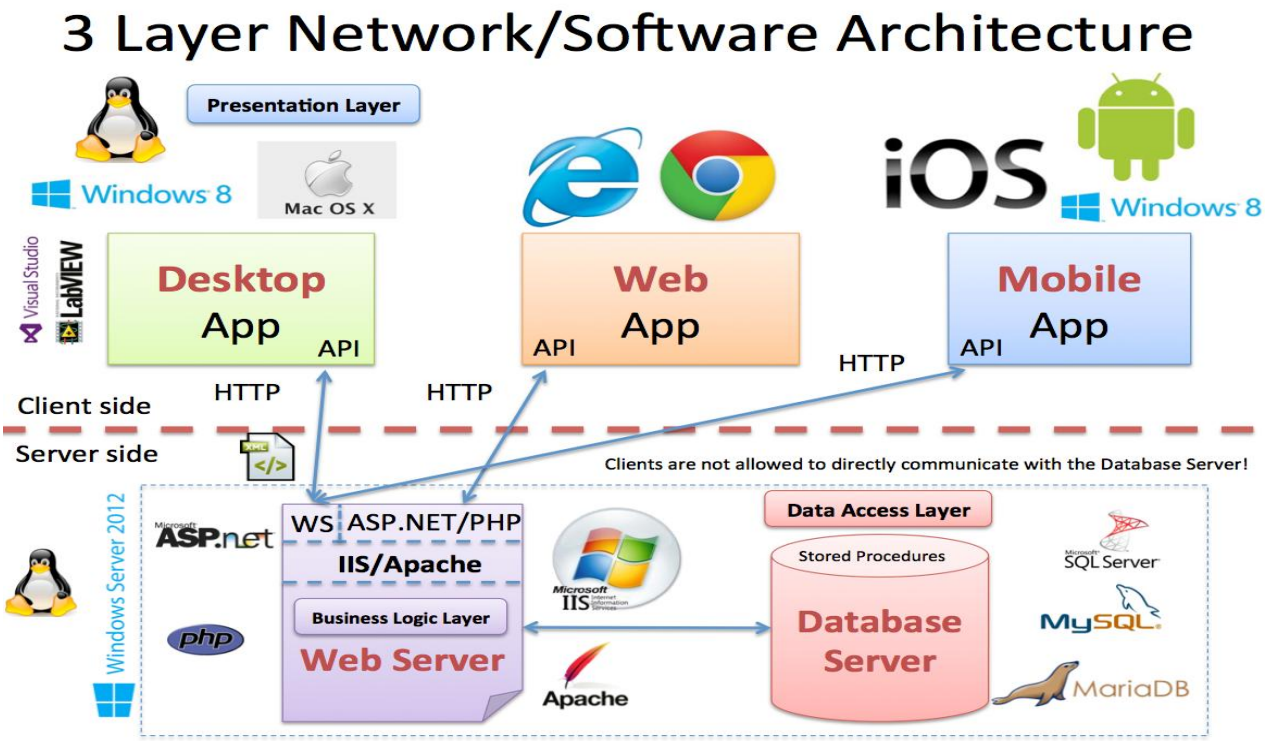


Figure 19-12: Software Architecture

A short description of the different layers:

Presentation Tier:

This is the topmost level of application. The presentation tier displays information related to such services as browsing merchandise, purchasing, and shopping cart contents. It communicates with other tiers by which it puts out the results to the browser/client tier and all other tiers in the network.

In simple terms, it is a layer which users can directly access such as a web page, or an operating systems GUI

Application tier:

(Other term used: Business logic, logic tier, data access tier, or middle tier) The application tier is pulled out from the presentation tier and, as its own layer. It controls an application's functionality by performing detailed processing.

Data tier:

This tier consists of database servers. Here information is stored and retrieved. This tier keeps data neutral and independent from application servers or business logic. Giving data its own tier also improves scalability and performance.

Figure 19-13 shows a sketch of the typical logical layers or tiers in a 3-tier/layer architecture.

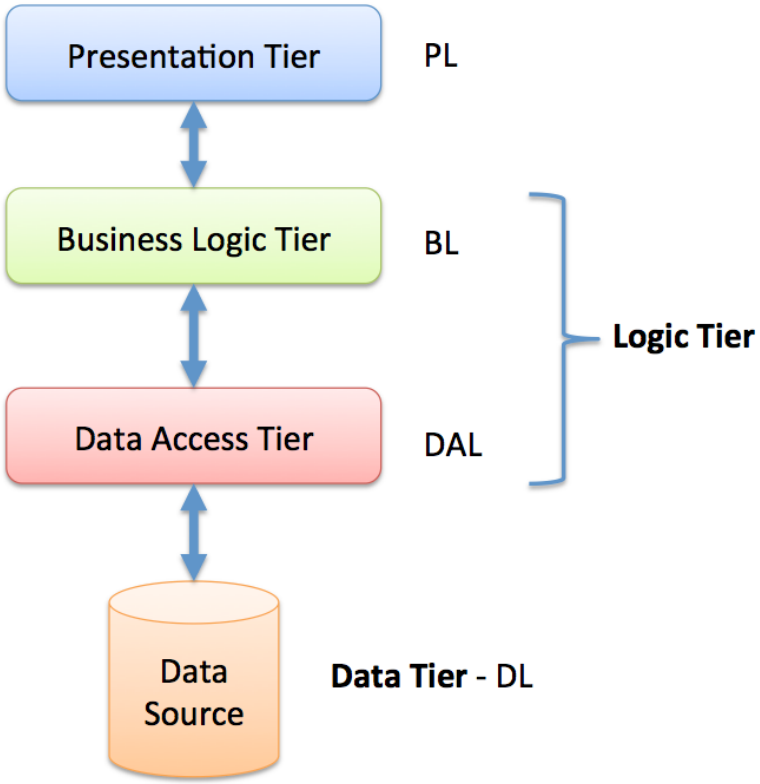


Figure 19-13: 3-tier Architecture

Figure 19-14 gives an overview of the 3-tier architecture and gives a short description of the different layers.

In web development, 3-tiers are often common, including the following

- A Web Server
- An Application Server
- A Database

Web Services could be used for communication between the different layers.

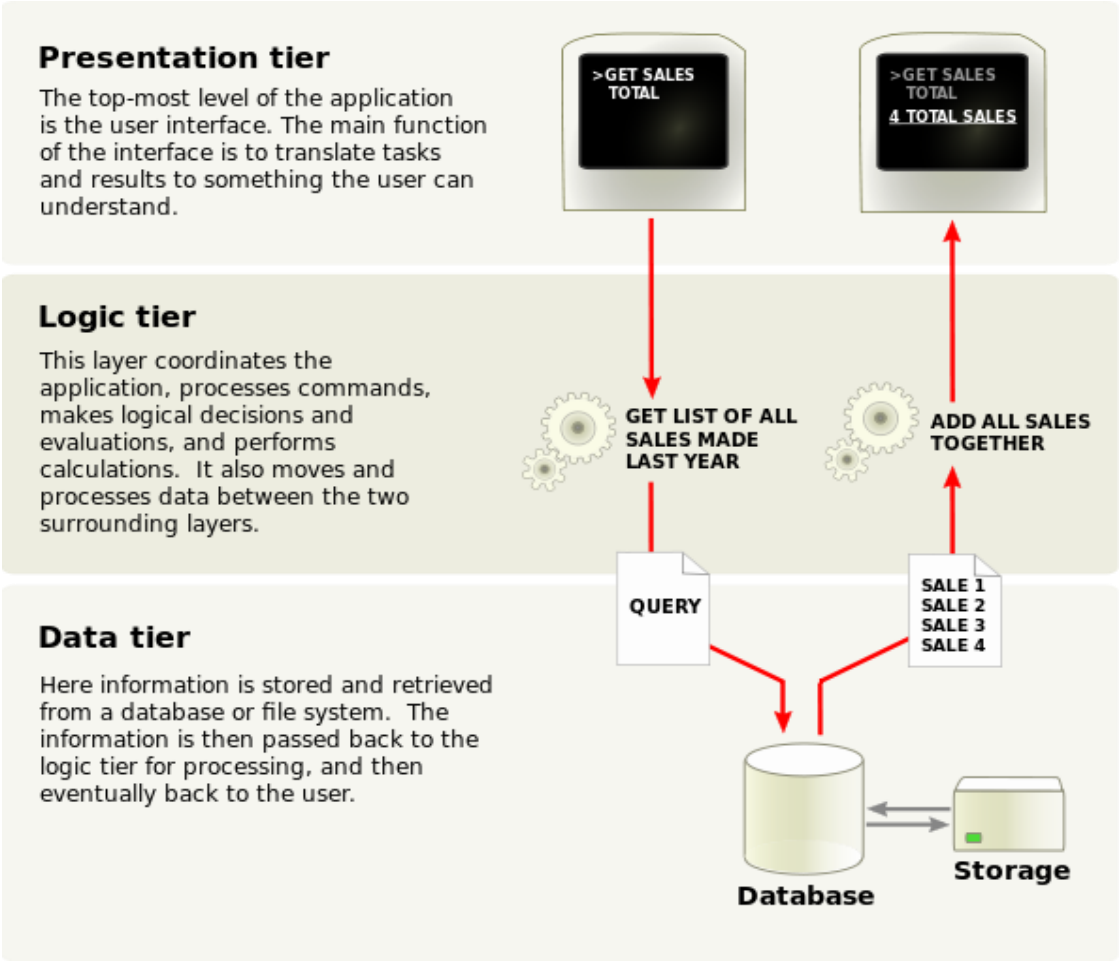


Figure 19-14: 3-tier Overview

In Figure 19-15 we see an example of 3-layer architecture software.

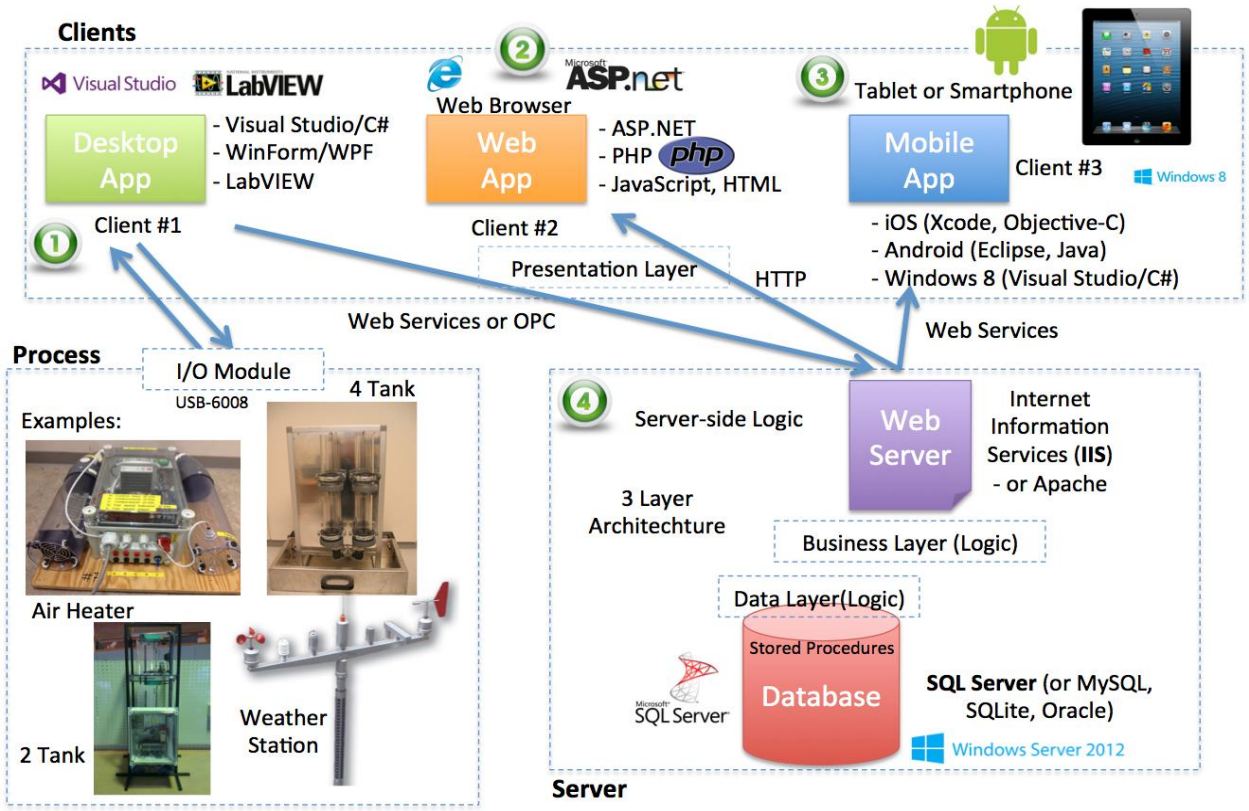


Figure 19-15: Example of 3-layer Architecture Software